

**Figura 13.144** Rutas de acceso para clases de otros paquetes

como cadena de nombre en el compartimento del nombre (Figura 13.144). Una ruta de acceso completa se especifica encadenando los nombres de paquete separados por dos signos de dos puntos (::). Varias clases de diferentes paquetes pueden utilizar el mismo nombre de clase, suponiendo que sus rutas de acceso son distintas, pero esta duplicación puede dar lugar a errores, por lo que debe evitarse en lo posible.

También aparecen referencias a clases en expresiones de texto, generalmente en especificaciones de tipo para atributos y variables. En expresiones de texto, una referencia a una clase se indica simplemente utilizando el propio nombre de la clase, incluyendo un posible nombre de paquete, pero sujeto a las reglas de sintaxis de la expresión.

## nombre de rol

Nombre de un extremo de asociación concreto dentro de una asociación.

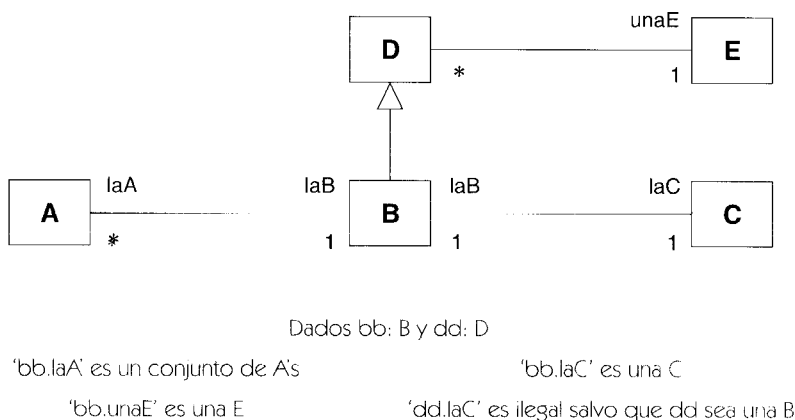
*Véase también* pseudoatributo.

## Semántica

Los nombres de rol proporcionan un nombre para identificar un extremo de asociación dentro de una asociación y también para navegar de un objeto a otro empleando la asociación. Dado que se puede utilizar un nombre de rol en estas dos formas complementarias, el nombre tiene que ser único en ambos espacios de nombres simultáneamente.

Todos los nombres de rol de una asociación tienen que ser diferentes. Dentro de una auto-asociación (una asociación que implica más de una vez a la misma clase), se necesitan los nombres de rol para deshacer la ambigüedad existente en los extremos que están conectados a una misma clase. En caso contrario, los nombres de rol son opcionales porque se pueden utilizar los nombres de las clases para deshacer la ambigüedad de los extremos.

También se utilizan los nombres de rol para navegar de un objeto a los objetos vecinos relacionados. Cada clase “ve” las asociaciones que están conectadas a ella y puede utilizarlas para hallar objetos relacionados con alguna de sus instancias. Por convenio, el nombre de rol de un extremo de asociación que está conectado a una clase vecina se emplea para formar una expresión de navegación con objeto de acceder al objeto o conjuntos de objetos que están relacionados por esa asociación. En la Figura 13.145, considérese la clase **B** que



**Figura 13.145** Navegación a través de asociaciones

está asociada a la clase **A** mediante una asociación uno-a-muchos y a la clase **C** mediante una asociación uno-a-uno. Dado un **bb** de clase **B**, la expresión **bb.laA** proporciona un conjunto de objetos de clase **A** y la expresión **bb.laC** proporciona un objeto de clase **C**. De hecho, el nombre de rol de lado lejano de la asociación es como un pseudoatributo de una clase, esto es, se puede utilizar como un término en una expresión de acceso para recorrer la asociación.

Dado que un nombre de rol se puede utilizar como un nombre de atributo para extraer valores, el nombre de rol entra en el espacio de nombres de la clase que se encuentra en el lado lejano de la asociación. Va al mismo espacio de nombres que los nombres de los atributos. Tanto los nombres de atributos como los nombres de rol tienen que ser únicos dentro de ese espacio de nombres. Los atributos y los nombres de roles de asociación se heredan; también los nombres de atributos y de pseudoatributos tienen que ser exclusivos entre los nombres heredados. Un nombre de rol asociado a una clase antecesora se puede utilizar para la navegación en un descendiente. En la Figura 13.145, la expresión **bb.unaE** es válida porque la clase **B** hereda el nombre de rol **unaE** de la clase **D**.

Los nombres de rol y los nombres de asociación son opcionales si se puede identificar de forma única a todas las asociaciones. Una asociación puede ser identificada bien por un nombre de asociación o bien por los nombres de rol que aparecen en sus extremos. No es necesario tenerlos todos, aunque está permitido. Si se trata de la única asociación existente entre dos clases, se puede omitir tanto el nombre de la asociación como los nombres de rol. En principio, se necesita un nombre de rol para una expresión de navegación. En la práctica, las herramientas pueden proporcionar una regla por defecto para crear nombres de rol implícitos a partir de los nombres de las clases asociadas.

## Notación

Los nombres de rol se muestran mediante una cadena gráfica situada cerca del extremo de una ruta de asociación, en el lugar en que ésta llega a un cuadro de clase. Si existe, el nombre de rol no puede suprimirse.

El nombre de rol puede llevar un marcador de visibilidad —una punta de flecha— que indica si el elemento del extremo lejano de la asociación puede o no ver al elemento asociado al nombre de rol.

## nombre de ruta

Cadena compuesta por la concatenación de los nombres de los espacios de nombres anidados que contienen a un elemento, partiendo del espacio de nombres implícito y anónimo que contiene todo el sistema y llegando al nombre del elemento en sí.

## Semántica

Los nombres de ruta identifican de modo único a los elementos del modelo, tales como un atributo o un estado, dentro del sistema. Se pueden utilizar en una expresión para hacer referencia a un elemento. No todas las clases de elementos poseen nombre.

## Notación

Los nombres de ruta se muestran como una lista de espacios de nombres anidados y de nombres de elementos separados mediante doble signos de dos puntos (::). Un espacio de nombres es un paquete o un elemento que posee declaraciones anidadas.

Contabilidad:: Personal:: Empleado:: dirección

El atributo **dirección** de la clase **Empleado** perteneciente al paquete **Personal** que forma parte de **Contabilidad**.

Los nombres de ruta son referencias de un elemento del paquete denotado por la ruta precedente.

## nota

Un símbolo adecuado para mostrar un comentario u otra información textual, tal como el cuerpo de un método o una restricción.

## Notación

Una nota es un rectángulo con una esquina doblada, concretamente la esquina superior derecha. Contiene un texto o un texto extendido (tal como un documento incrustado) que no será interpretado por UML. Las notas pueden presentar información de diferentes tipos respecto a los elementos del modelo; puede tratarse de un comentario, una restricción o un método. Las notas no suelen indicar explícitamente el tipo de elemento que representan pero esto suele ser evidente a partir de su forma y del modo en que se utilizan. Dentro de una herramienta de mo-

delado, el elemento subyacente quedará explícito en el modelo. Se puede asociar la nota al elemento que describe empleando una línea discontinua. Si la nota describe múltiples elementos, se lleva una línea discontinua hasta todos y cada uno de esos elementos.

Las notas pueden tener una palabra reservada entre comillas para indicar su significado. La palabra clave «**constraint**» denota una restricción.

### Ejemplo

La Figura 13.146 muestra notas que se usan con distintos fines, incluyendo una restricción que afecta a una operación, una restricción que afecta a una clase y un comentario.

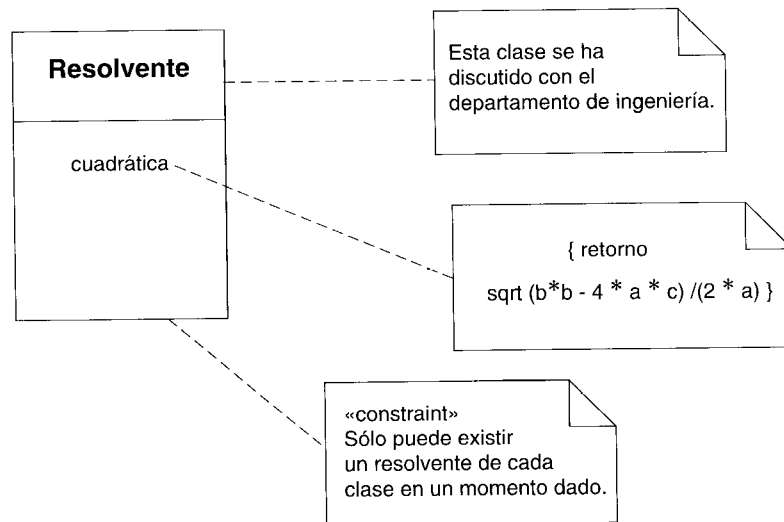


Figura 13.146 Notas

## notación canónica

UML define una notación canónica que utiliza dibujos de líneas monocromáticas para visualizar todos los modelos. Éste es el “formato de publicación” estándar de los modelos en UML, aplicable también a los diagramas impresos.

Las herramientas de edición gráfica pueden extender la notación a conveniencia y proporcionar nuevas capacidades interactivas. Por ejemplo, una herramienta puede proporcionar la capacidad de resaltar en pantalla los elementos seleccionados. Otras capacidades de interacción incluyen la posibilidad de navegación dentro del modelo, y el filtrado de elementos visualizados dependiendo de unas propiedades seleccionadas. Este tipo de formato es efímero y no viene impuesto por UML. En una visualización interactiva existe un pequeño riesgo de ambigüedad, por lo que el usuario debe poder pedir explicaciones aclarativas. Por todo ello, UML hace hincapié en la forma canónica impresa, que toda herramienta debe ofrecer, a sabiendas de que una herramienta interactiva puede y debe proporcionar extensiones interactivas.

## número de secuencia

Parte textual de la etiqueta de un mensaje en un diagrama de colaboración que indica el orden relativo de ejecución de los mensajes en una interacción. Un número de secuencia puede mostrar la localización del mensaje dentro de una secuencia de llamadas anidadas, el nombre de un hilo de control y una especificación de ejecución condicional e iterativa.

*Véase* colaboración, mensaje.

## objeto

Entidades discretas, con límites bien definidos y con identidad, que encapsulan el estado y el comportamiento; se dice también de las instancias de una clase.

*Véase también* clase, identidad, instancia.

## Semántica

Un objeto es una instancia de una clase, que describe el conjunto de posibles objetos que pueden existir. Un objeto se puede ver desde dos perspectivas relacionadas: como una entidad en un determinado instante de tiempo que posee un valor específico y como un poseedor de identidad que tiene distintos valores a lo largo del tiempo. El primer punto de vista es adecuado para una instantánea, que representa al sistema en un cierto instante de tiempo. Un objeto de una instantánea tiene una localización (en un sistema distribuido) así como valores para todos sus atributos. Los objetos están asociados a un conjunto de enlaces que los conectan con otros objetos.

Cada objeto posee su propia identidad exclusiva y se puede hacer referencia a él mediante una denominación exclusiva que lo identifica y permite acceder a él. La vista de los objetos como identidades es adecuada para una instancias de colaboración, en la cual el objeto posee relaciones de tiempo de ejecución con otros objetos y las usa para intercambiar instancias de mensajes.

Los objetos contienen una ranura de atributo para cada atributo de su descriptor completo, esto es, para todos los atributos declarados en su clase directa y en todas las clases antecesoras. Cuando ha finalizado la instanciación e iniciación de un objeto, cada ranura contiene un valor que es una instancia del clasificador declarado como tipo del atributo. Cuando se ejecuta el sistema, el valor de la ranura de atributo puede cambiar salvo que la propiedad de modificabilidad de atributos se lo impida. Los valores del objeto deben satisfacer todas las restricciones implícitas y explícitas impuestas por el modelo y deben hacerlo en todos los momentos situados entre la ejecución de operaciones. Durante la ejecución de una operación, se pueden violar temporalmente las restricciones.

Si se admite la clasificación múltiple en un entorno de ejecución, entonces un objeto puede ser instancia directa de más de una clase. El objeto contiene una ranura de atributo para cada atributo que se haya declarado en cualquiera de sus clases directas o en cualquiera de sus antecesores. El mismo atributo no podrá aparecer más de una vez, pero si dos clases directas

son descendientes de un antecesor común entonces sólo se hereda una copia de cada atributo del antecesor, independientemente de la multiplicidad de rutas que lleven a él.

Si se admite la clasificación dinámica, el objeto puede cambiar su clase directa durante la ejecución. Si se ganan atributos en el proceso, entonces es preciso especificar sus valores mediante la operación que cambia la clase directa.

Si se permiten tanto la clasificación múltiple como la clasificación dinámica, entonces el objeto puede ganar o perder clases directas durante la ejecución. Sin embargo, el número de clases directas nunca puede ser menor que uno (tiene que tener alguna estructura, aun cuando sea transitoria).

Se puede invocar a un objeto para que ejecute cualquier operación que aparezca en el descriptor completo de cualquier clase directa, esto es, que posee operaciones heredadas tanto directa como indirectamente.

Los objetos se pueden utilizar como valor de cualquier variable o parámetro cuyo tipo declarado sea de la misma clase, o un antecesor de la clase directa del objeto. En otras palabras, una instancia de cualquier descendiente de una clase puede aparecer como valor de una variable cuyo tipo declarado sea precisamente esa clase. Éste es el principio capacidad de sustitución. Este principio no es una necesidad lógica, sino que existe para simplificar la implementación de lenguajes de programación.

## Notación

Todo objeto es una instancia de alguna clase. La regla general para la notación de instancias consiste en utilizar el mismo símbolo geométrico que el descriptor, subrayando el nombre de la instancia para distinguirla como individuo. En la instancia se muestran los posibles valores pero las propiedades compartidas por todas las instancias sólo se ponen de manifiesto en el descriptor.

La notación canónica de un objeto es un rectángulo con dos compartimentos. El compartimento superior contiene el nombre del objeto y el de la clase; el compartimento inferior contiene una lista de nombres y valores de atributos (Figura 13.147). No hay necesidad de mostrar operaciones porque son las mismas para todos los objetos de la clase.

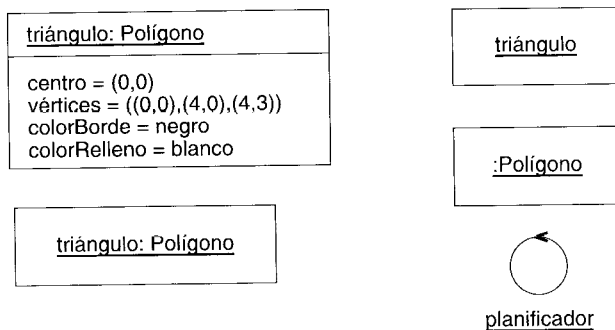


Figura 13.147 Notación de los objetos

El compartimento superior muestra el nombre del objeto y de su clase, ambos subrayados, empleando la sintaxis:

nombre-del-objeto : nombre-de-la-clase

El nombre de la clase puede incluir la ruta de acceso completa del paquete que la contiene, si es necesario. Los nombres de paquete preceden al nombre de la clase y se separan mediante grupos de dos signos de dos puntos. Por ejemplo:

ventanaVisualización:SistemaVentanas:: VentanasGraficas::Ventana

Se puede mostrar textualmente un estereotipo para la clase (entre comillas y por encima del nombre de la clase) o bien como un icono en la esquina superior derecha. El estereotipo del objeto tiene que coincidir con el estereotipo de su clase.

Para mostrar múltiples clases de las que el objeto sea una instancia, haga uso de una lista de nombres de clases separadas mediante comas. Algunas de las clases pueden ser roles transitorios que desempeña el objeto durante una colaboración. Por ejemplo:

unaPersona: Profesor, Esquiador

Para mostrar la presencia de un objeto en un estado particular de la clase, utilice la sintaxis:

nombre-de-objeto : nombre-de-clase [ lista-de-nombres-de-estado ]

La lista tiene que ser una lista de nombres de estado separados mediante comas y los estados tienen que poder producirse concurrentemente de forma legal.

Para mostrar un cambio de clase (clasificación dinámica) es preciso mostrar dos veces el objeto, una con cada clase. Los dos símbolos se conectan mediante una relación de conversión para mostrar que representan un mismo objeto.

El segundo compartimento muestra los atributos del objeto y sus valores en forma de lista. Cada línea de valor tiene la sintaxis:

nombre-de-atributo : tipo = valor

El tipo es redundante con la declaración del atributo en la clase y se puede omitir. El valor se especifica como una cadena que representa el valor. Los nombres de los atributos no se subrayan.

Se puede omitir el nombre del objeto. En este caso, deben mantenerse los dos puntos con el nombre de la clase. Esto representa un objeto anónimo de esa clase, al cual darán una identidad sus relaciones. Todo símbolo que contiene un objeto anónimo denota un objeto individual que se distingue por sus relaciones con otros objetos.

Se puede suprimir la clase del objeto (junto con los dos puntos) pero debería mostrarse siempre que fuera posible, para no inducir a confusión.

El compartimento de valores de atributos, en bloque, puede suprimirse.

Se pueden suprimir aquellos atributos cuyos valores no sean relevantes.

Para mostrar los cambios de valor de los atributos durante una computación, muestre dos versiones del mismo objeto con una relación de conversión entre ellos.

## objeto activo

Objeto que tiene un hilo de control y que puede iniciar una actividad de control; instancia de una clase activa.

*Véase también* objeto pasivo, proceso, hilo.

## Semántica

Un objeto activo no se ejecuta dentro de otro hilo, marco de pila, o máquina de estados. Tiene un lugar de control independiente dentro de la ejecución global de un sistema. En cierto sentido, *es* el hilo. Cada objeto activo es un lugar de ejecución distinto. Los objetos activos no son reentrantes, y además la ejecución recursiva no es posible sin la creación de objetos adicionales.

Un objeto activo es la raíz de la pila de ejecución en términos de computación convencional. La creación de un objeto activo inicia una nueva instancia de una máquina de estados. Cuando la máquina de estados realiza una transición, se crea un marco de pila de ejecución que continúa hasta que la acción de la transición se ejecuta completamente y el objeto espera una entrada externa. Por tanto, un objeto activo no se ejecuta en el ámbito de otro objeto; puede ser creado por una acción de otro objeto, pero una vez creado, tiene una existencia independiente. El creador puede ser un objeto activo o pasivo. Un objeto activo está dirigido por eventos, y las operaciones que otros objetos realicen sobre él deberían aparecer en el objeto activo como eventos de llamada.

Se puede crear un objeto pasivo como parte de una acción de otro objeto. Un objeto pasivo tiene su propio espacio de direcciones, pero no tiene ningún hilo de control. Sus operaciones se pueden llamar desde el bloque de pila de un objeto activo, y puede ser modelado como una máquina de estados para mostrar los cambios de estado provocados por las operaciones realizadas sobre él.

Un proceso de un sistema operativo convencional es muy similar a un objeto activo. Un hilo de un sistema operativo puede implementarse o no utilizando un objeto activo.

La distinción entre activo y pasivo es principalmente una decisión de diseño y no restringe la semántica de los objetos. Tanto los objetos activos como los pasivos pueden tener máquinas de estados e intercambiar eventos.

## Notación

Un rol de colaboración para un objeto activo se representa en un diagrama de colaboración mediante un rectángulo con un borde más grueso. Es frecuente representar los roles de objeto activo como compuestos con partes empotradas.

Un objeto activo también se representa mediante un símbolo de objeto con el borde más grueso, y con el nombre subrayado, pero los objetos activos aparecen sólo en ejemplos de ejecución y por tanto no son muy comunes.



Para indicar que un objeto es activo, también se puede utilizar la palabra clave **{active}** en las propiedades.

## Ejemplo

La Figura 13.148 muestra tres objetos activos de un sistema de automatización de una fábrica: un robot, un horno y un gestor de la fábrica, que es un objeto de control. Los tres objetos existen y se ejecutan concurrentemente. El gestor de la fábrica inicia un hilo de control en el paso 1, que más tarde se divide en dos hilos de control concurrentes, A1 y B1, ejecutados por el horno y el robot respectivamente. Cuando ambos terminan su ejecución, los hilos se juntan (paso 2) en el gestor de la fábrica. Cada objeto sigue vivo y conserva su estado hasta que le llegue el próximo evento.

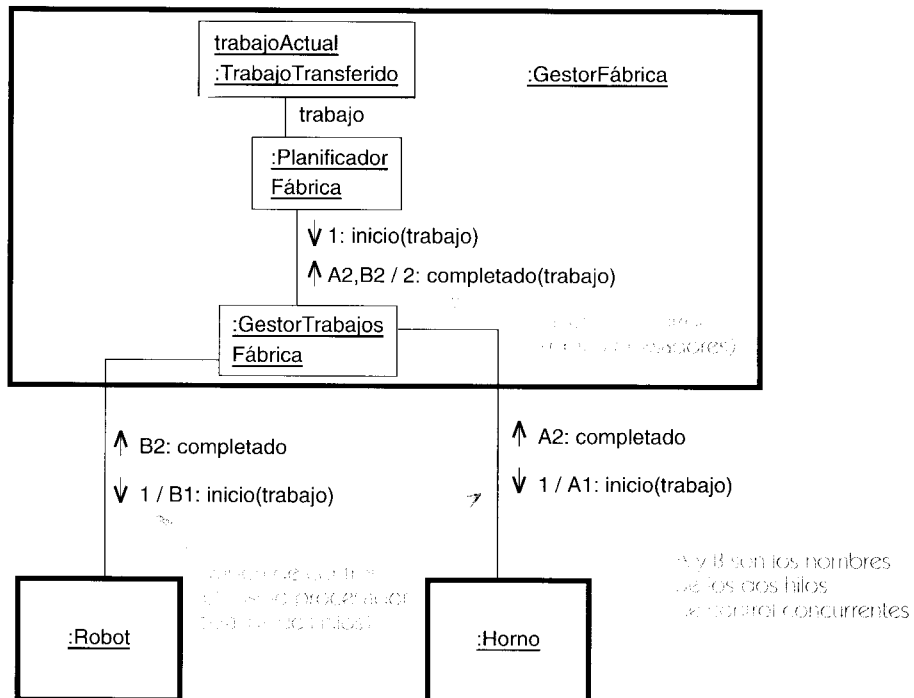


Figura 13.148 Colaboración con objetos activos y control concurrente

## objeto compuesto

Un objeto compuesto representa un objeto de alto nivel construido con piezas unidas firmemente. Es una instancia de una clase compuesta, lo que implica la relación de agregación entre la clase y sus partes. Un objeto compuesto es similar a (pero más simple y más restrictivo que) una colaboración. Sin embargo, está definido por composición en un modelo estático, en lugar de por las relaciones dependientes de contexto de una colaboración.

Véase también composición.

## Semántica

Un objeto compuesto tiene una relación de composición con todas sus piezas. Esto significa que es responsable de su creación y destrucción, y que no hay otro objeto que sea responsable en la misma medida. Es decir, no hay que preocuparse de la recolección de basura en lo referente a las piezas; el objeto compuesto puede y debe destruirlas cuando muere, o bien debe entregar esta responsabilidad a otro objeto.

La relación de composición se implementa a menudo por contención física dentro de la misma estructura de datos que el propio objeto compuesto (generalmente un registro). La contención física asegura que el tiempo de vida de las piezas se ajusta al tiempo de vida del objeto compuesto.

## Notación

Una red de objetos y de enlaces se puede anidar con un compartimento gráfico dentro de un símbolo de objeto. El compartimento gráfico se muestra como un compartimento añadido debajo del compartimento de los atributos (el compartimento de los atributos se puede suprimir). Los objetos y los enlaces contenidos en la región gráfica son partes compuestas del objeto compuesto. Sin embargo, un enlace cuya línea rompe el límite del objeto, no es una parte compuesta de él; es un enlace entre los objetos separados.

## Ejemplo

La Figura 13.149 muestra un objeto compuesto, llamada ventana de escritorio, integrada por varias partes. Contiene múltiples instancias de la clase de **Barra de Desplazamiento**. Cada

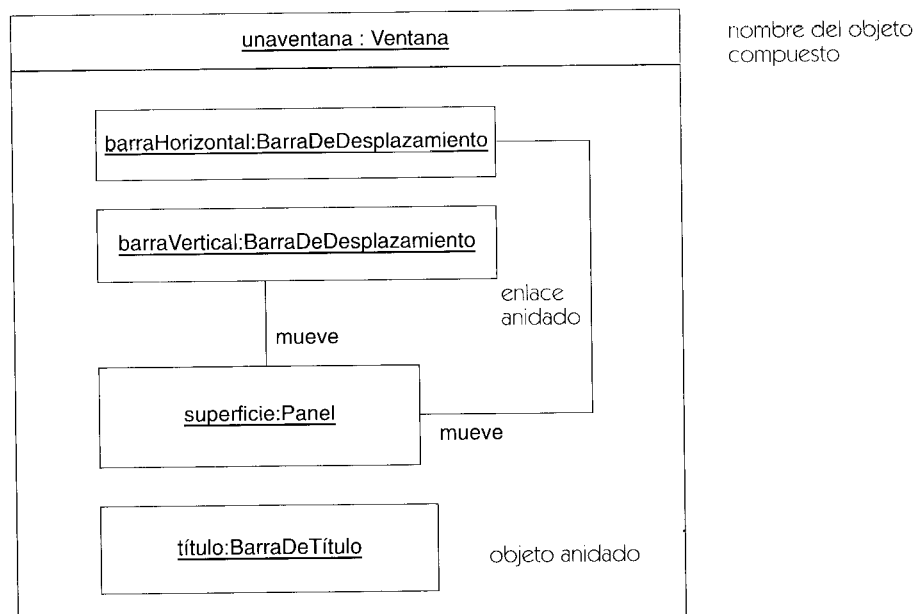


Figura 13.149 Objeto compuesto

instancia tiene su propio nombre y rol dentro del objeto compuesto. Por ejemplo, las **Barras Horizontales** y las **Barras Verticales** son ambas **Barras de Desplazamiento**, pero se comportan de forma diferente dentro del objeto compuesto. A este respecto, son como roles de colaboración.

## objeto pasivo

Objeto que no posee su propio hilo de control. Sus operaciones se ejecutan en un hilo de control basado en un objeto activo.

## Semántica

Diremos que un objeto activo es aquel que posee un hilo de control y que puede iniciar la actividad de control. Un objeto pasivo será aquel que posea valor por no puede iniciar el control. Sin embargo, una operación de un objeto pasivo puede enviar mensajes cuando procesa una solicitud recibida de un hilo ya existente.

## Notación

Los objetos pasivos se representan mediante un rectángulo de clase que tiene subrayado en nombre del objeto. Las clases pasivas se representan mediante un rectángulo de clase en que el nombre de la clase no está subrayado. El rectángulo tiene un borde normal (no está acentuado en negrita). Los objetos o clases activos se dibujan con un borde acentuado.

## objeto persistente

Objeto que sigue existiendo después de que ha dejado de existir el hilo que lo ha creado.

## objeto transitorio

Objeto que existe únicamente durante la ejecución del hilo que lo ha creado.

## OCL

Lenguaje de Restricción de Objetos (*Object Constraint Language*), un lenguaje para especificar restricciones y consultas. OCL no está destinado a escribir acciones o código ejecutable. Para una definición completa, véase el libro [Warmer-99].

## Semántica

El Lenguaje de Restricción de Objetos (OCL) es un lenguaje de texto para escribir expresiones de navegación, expresiones booleanas y otras consultas. Se puede emplear con objeto de construir expresiones para restricciones, condiciones de guarda, acciones, precondiciones y postcondiciones, aserciones y otras clases de expresiones UML. Se hallará una descripción completa de la sintaxis y semántica de OCL en [Warmer-99]. El resumen seleccionado que se muestra a continuación contiene la sintaxis de OCL más útil para crear expresiones de navegación y condiciones booleanas. El lenguaje completo contiene un gran número de operadores predefinidos, aplicables a colecciones y a tipos predefinidos.

## Notación

La sintaxis de algunas expresiones comunes de navegación se muestra más abajo. Estas formas se pueden encadenar. El elemento situado más a la izquierda debe ser una expresión de un objeto o conjunto de objetos. Las expresiones están destinadas a funcionar en conjuntos de valores cuando proceda. Para más detalles y para disponer de toda la sintaxis, consúltese la descripción de OCL.

`elemento .selector`

`selector` es el nombre de un atributo del elemento o el nombre de un rol del extremo blanco de un enlace asociado al elemento. El resultado es el valor del atributo o del objeto u objetos relacionados. El resultado es un valor o un conjunto de valores, dependiendo de las multiplicidades del elemento y de la asociación.

`elemento .selector (expresión booleana)`

`selector` es el nombre de una operación que se aplica al elemento. El resultado es el valor proporcionado por la operación aplicada al elemento.

`elemento .selector [valor calificador]`

`selector` denota una asociación calificada que califica al elemento. `valor-calificador` es un valor para el atributo calificador. El resultado es el objeto relacionado que seleccione el calificador. Obsérvese que esta sintaxis es aplicable a la indexación de matrices como una forma de calificación.

`conjunto -> propiedad`

`propiedad-conjunto` es el nombre de una función de conjuntos propia de OCL. El resultado es la propiedad del conjunto. No es válido si `propiedad-conjunto` no es un función predefinida de OCL. Se enumeran más adelante varias de las propiedades.

`conjunto -> select (expresión booleana)`

`expresión-booleana` está escrita en términos de los objetos que pertenecen al conjunto. El resultado es el subconjunto de objetos del conjunto para los cuales es verdadera la expresión booleana.

|                  |                                                                                 |
|------------------|---------------------------------------------------------------------------------|
| conjunto -> size | El número de elementos que hay en el conjunto.                                  |
| self             | Denota el objeto actual (se puede omitir si el contexto queda claro).           |
| operadores       | Los operadores aritméticos y booleanos habituales<br>= < > <= >= <> + - * / not |

## Ejemplo

```
vuelo.piloto.horas_entrenamiento >= vuelo.avión.mínimo_horas
// el piloto entrenado tiene suficientes horas de entrenamiento

compañía.empleados ->select (cargo = "Jefazo" y self.informes ->size > 10)>
// el jefe de la flota que tiene mas de diez informes
```

## operación

Una operación es la especificación de una transformación o consulta que puede tener que ejecutar un objeto. Posee un nombre y una lista de parámetros. Un método es un procedimiento que implementa una operación. Posee la descripción de un algoritmo o procedimiento. Una operación de una clase activa también se puede implementar utilizando un evento de llamada.

*Véase también* llamada, evento de llamada, método.

## Semántica

La operación especifica una transformación del estado del objeto destino (y posiblemente del estado del resto del sistema que se pueda alcanzar desde el objeto destino) o bien una consulta que proporciona un valor a quien invoque esa operación. Las operaciones se pueden implementar como métodos o como un evento de llamada que da lugar a una transición en la máquina de estados del objeto activo. Las operaciones se invocan mediante una llamada, que deja en suspenso a su emisor hasta que finalice la ejecución de la operación, tras lo cual el emisor recupera el control tras el punto en que se hiciera la llamada, recibiendo un valor de retorno si es que la operación produce uno.

Las operaciones se declaran en una clase. La declaración es heredada por los descendientes de la clase. Si otra declaración tiene la misma "signatura coincidente", entonces se trata de la misma operación. Una implementación puede especificar una regla de firmas coincidentes para detectar posibles conflictos, pero por defecto se incluye el nombre de la operación y de las clases (pero no los nombres o direcciones) de los parámetros, sin incluir los parámetros que se devuelven al retornar. Esa misma operación podría aparecer en una clase descendiente. En tal caso, será tratada como una repetición de la declaración heredada y se ignora. El propósito es permitir que una operación se puede declarar múltiples veces en clases desarrolladas en diferentes paquetes, empleando la coincidencia de nombres. La declaración de la operación que sea el antepasado común de todas las declaraciones de esa clase es lo que se denomina el origen (según Bertrand Meyer). Representa la declaración vigente de la operación, que es heredada por las demás.

Si dos declaraciones de una operación tienen igual nombre y la misma lista ordenada de tipos de parámetros (sin incluir los parámetros de retorno) pero varían otras propiedades (por ejemplo, un parámetro es de entrada en una operación pero de salida en otra) entonces las declaraciones tienen un conflicto y el modelo está mal formado.

Un método es la implementación de una operación (también puede estar implementado mediante un evento de llamada). Si se declara una operación en una clase sin la propiedad abstracta, entonces tiene una definición de método en la clase. En caso contrario, la operación puede ser abstracta (y no existe un método) o bien puede ser concreta, con un método heredado.

## **Estructura**

Las operaciones poseen los siguientes componentes principales:

|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| concurrency       | Es la semántica de llamadas concurrentes a una misma instancia pasiva, una enumeración. Los valores posibles son:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>sequential</b> | Quiénes la invoquen deben coordinarse de modo que sólo se pueda ejecutar una llamada a un objeto (para cualquier operación secuencial) al mismo tiempo. Si se producen llamadas concurrentes, entonces no se puede garantizar la semántica y la integridad del sistema.                                                                                                                                                                                                                                                                                                                                          |
| <b>guarded</b>    | Pueden producirse múltiples llamadas simultáneas procedentes de hilos concurrentes a un mismo objeto (para cualquier operación guardada) pero sólo se permite que comience una de cada vez. Las demás quedan bloqueadas hasta que finalice la ejecución de la primera operación. Es responsabilidad del modelador asegurarse de que no se produzcan interbloqueos como consecuencia de bloqueos simultáneos. Las operaciones guardadas deben comportarse correctamente (o deben bloquearse a sí mismas) si se produce una operación secuencial simultánea bien no se puede dar por válida la semántica guardada. |
| <b>concurrent</b> | Pueden producirse múltiples llamadas procedentes de hilos concurrentes a un único objeto (para operaciones concurrentes). Todas ellas pueden desarrollarse concurrentemente con una semántica correcta. Las operaciones concurrentes deben diseñarse de tal modo que actúen correctamente si se produjera una operación concurrente, secuencial o guardada en ese mismo objeto. En caso contrario, no se puede dar por válida la semántica concurrente.                                                                                                                                                          |
| polimorfismo      | Indica si la implementación de la operación (el método o evento de llamada) puede o no ser anulado por las clases descendientes. Si es así, la implementación puede ser anulada por una clase descendiente que proporcione una nueva definición del método o una transición diferente de la máquina de estados. La implementación adopta distintas formas—esto es, es polimórfica—. En caso contrario, la implementación actual será heredada sin cambios por todos los descendientes. Posee una única forma.                                                                                                    |

|                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                 |                                                       |              |                                                                                                               |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|-------------------------------------------------------|--------------|---------------------------------------------------------------------------------------------------------------|
| consulta            | Indica si la ejecución de la operación deja o no intacto el estado del sistema; esto es, si se trata o no de una consulta. En caso afirmativo, la operación proporciona un valor pero no posee efectos secundarios. En caso contrario, puede alterar el estado del sistema, pero no se garantiza que haya algún cambio.                                                                                                                                                                                                                                                                                                       |                 |                                                       |              |                                                                                                               |
| nombre              | Es el nombre de la operación, una cadena. El nombre, junto con la lista de tipos de parámetros (sin incluir los nombres o tipos de los parámetros de retorno) es lo que se denomina <i>signatura coincidente</i> de la operación. La <i>signatura coincidente</i> tiene que ser exclusiva dentro de la clase y de sus antecesores. Si existe un duplicado, se toma como una <i>redeclaración</i> de la operación, que tiene que coincidir perfectamente. Si coinciden, entonces se ignoran todas las declaraciones de la operación salvo al de su antecesor más remoto. Si no coinciden, entonces el modelo está mal formado. |                 |                                                       |              |                                                                                                               |
| lista de parámetros | Es la lista de declaraciones de los parámetros de esa operación. Véase <i>lista de parámetros</i> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                 |                                                       |              |                                                                                                               |
| tipos de retorno    | Se trata de una lista formada por los tipos de los valores proporcionados por una llamada a esa operación. Si la operación no proporciona valores, entonces esta propiedad tiene el valor null. Observe que hay muchos lenguajes que no admiten proporcionar múltiples valores, pero sigue siendo un concepto válido de modelado que se puede implementar de diferentes maneras, tales como tratar a uno o más de los parámetros como valores de salida.                                                                                                                                                                      |                 |                                                       |              |                                                                                                               |
| alcance             | Denota si la operación es aplicable a objetos individuales o a la clase en sí (alcance de propietario). Los valores posibles son: <table data-bbox="277 1075 1306 1197"> <tr> <td><b>instance</b></td><td>La operación se puede aplicar a objetos individuales.</td></tr> <tr> <td><b>class</b></td><td>La operación se puede aplicar a la clase en sí, por ejemplo una operación que crea una instancia de la clase.</td></tr> </table>                                                                                                                                                                                      | <b>instance</b> | La operación se puede aplicar a objetos individuales. | <b>class</b> | La operación se puede aplicar a la clase en sí, por ejemplo una operación que crea una instancia de la clase. |
| <b>instance</b>     | La operación se puede aplicar a objetos individuales.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                 |                                                       |              |                                                                                                               |
| <b>class</b>        | La operación se puede aplicar a la clase en sí, por ejemplo una operación que crea una instancia de la clase.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                 |                                                       |              |                                                                                                               |
| especificación      | Denota una expresión que describe los efectos producidos al ejecutar la operación, por ejemplo, una <i>precondición</i> o <i>postcondición</i> . El formato de la especificación no lo prescribe UML y puede adoptar distintas formas.                                                                                                                                                                                                                                                                                                                                                                                        |                 |                                                       |              |                                                                                                               |
| visibilidad         | Es la visibilidad de la operación por parte de clases distintas de aquella que la define. Véase <i>visibilidad</i> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                 |                                                       |              |                                                                                                               |

Los métodos poseen los mismos componentes que las operaciones. Además, pueden tener uno o más de los siguientes:

|                |                                                                                                                                                                                                                                                                       |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| comportamiento | Se trata de una máquina de estados opcional que describe la implementación del método.                                                                                                                                                                                |
| cuerpo         | Es una expresión que describe el procedimiento del método. Puede estar representada por una cadena o posiblemente tenga un formato analizado sintácticamente. Normalmente se expresa en un lenguaje de programación aunque se puede emplear una expresión en lenguaje |

|              |                                                                                                                                             |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------|
|              | natural en especificaciones informales. En general este valor no se proporciona si se proporciona la máquina de estados.                    |
| colaboración | Denota un conjunto de colaboraciones que describen la implementación del método como un conjunto de mensajes entre roles (una interacción). |

Los eventos de llamada tienen los mismos componentes que una operación. La implementación de la operación tiene que ser especificada por una o más de las transiciones que tienen a ese evento de llamada como disparador.

## Notación

Las operaciones se representan mediante una cadena de texto que se puede analizar sintácticamente descomponiéndose en propiedades de la operación. La sintaxis por defecto es

```
«stereotype» «visibilidad» nombre ( «lista-parámetros» ) : «tipo-proporcionado» { «cadena-de-propiedades» }
```

El estereotipo, la visibilidad, la expresión del tipo proporcionado y la cadena de propiedades son opcionales (junto con sus delimitadores). La lista de parámetros puede estar vacía. La Figura 13.150 muestra algunas operaciones típicas.

**Nombre.** Es una cadena que denota el nombre de la operación (sin incluir los parámetros).

**Lista de parámetros.** Denota una lista de declaraciones de parámetros separadas mediante comas, cada una de las cuales consta de una dirección, un nombre y un tipo. Toda la lista va encerrada entre paréntesis (incluyendo las listas vacías). Véase lista de parámetros y parámetros para disponer de todos los detalles.

**Tipo de retorno.** Se trata de una cadena que contiene una lista separada por comas de los nombres de los clasificadores (clases, tipos de datos o interfaces). La cadena de tipo va después del signo de dos puntos (:) que sigue a la lista de parámetros de la operación. Los dos puntos y la cadena de tipo proporcionado se omiten si la operación no proporciona valor alguno (por ejemplo, con el caso de **void** en C++). Algunos lenguajes de programación, aunque no todos, admiten múltiples valores de retorno.

**Visibilidad.** La visibilidad se representa mediante uno de los signos de puntuación '+', '#' y '-' que denota respectivamente **pública**, **protegida** o **privada**. Alternativamente, se puede mostrar la visibilidad como palabra reservada dentro de la cadena de propiedades (por ejemplo, { **visibilidad=privada** }). Es preciso utilizar esta forma para opciones definidas por el usuario o dependientes del lenguaje.

**Método.** Las operaciones y los métodos se declaran empleando la misma sintaxis. La aparición más externa de la signatura de una operación dentro de una jerarquía de generalización es

```
+visualizar (): Localización
+ocultar ()
«constructor» +crear ()
-asociarVentanaX(venX:ventanaX*)
```

**Figura 13.150** Lista de operaciones, con una gama de operaciones



la declaración de la operación. Las firmas idénticas en clases descendientes son declaraciones redundantes de la operación, pero pueden ser útiles para declarar operaciones cuando las clases se desarrollan por separado. Si la declaración de una operación tiene la propiedad abstracta (lo cual se denota poniendo en cursiva el nombre de la operación o la palabra reservada **abstract**) entonces no existe un método correspondiente a la declaración. En caso contrario, la declaración representa a la vez la declaración de una operación y un método que la implementa.

En las operaciones y métodos coincidentes se utiliza el nombre de la operación y la lista ordenada de tipos de parámetros, sin incluir los parámetros de retorno. Si las propiedades restantes no son consistentes (por ejemplo, si un parámetro de entrada coincide con un parámetro de salida) entonces existe un conflicto y el modelo está mal formado.

Si dos declaraciones de operación idénticas no tienen una declaración de operación en un antecesor común, pero las hereda una clase común, entonces el modelo está mal formado. En esta situación, las declaraciones producirían un conflicto en la clase que herede a ambas.

**Cuerpo del método.** Se puede mostrar el cuerpo de un método como una cadena situada dentro de una nota asociada a la declaración de una operación. El texto de la especificación debería ir encerrado entre llaves si se trata de una especificación formal en algún lenguaje (una restricción semántica). En caso contrario, debería ser un texto normal si es tan sólo una descripción en lenguaje natural del comportamiento (un comentario). La conexión de una declaración de método a su máquina de estados o a su colaboración carece de representación visual, pero en general se representaría mediante un hipervínculo dentro de una herramienta de edición.

**Especificación.** Se trata de una expresión que describe los efectos de llevar a cabo la operación. Éstos se pueden enunciar de diferentes maneras, incluyendo el texto, las precondiciones y postcondiciones y los invariantes. En todo caso, la especificación debería expresarse en términos de los efectos observables de la operación sobre el estado del sistema y no en términos del algoritmo de ejecución. El algoritmo es problema del método.

La especificación se muestra mediante una cadena de restricción situada dentro de una nota asociada a la operación de entrada.

**Consulta.** La opción se muestra mediante una cadena de propiedades de la forma **isQuery=true** o bien **isQuery=false**. La posibilidad **true** también se puede mostrar mediante la palabra reservada **query**. La ausencia de una opción explícita indica la opción **false**, esto es, que la operación puede alterar el estado del sistema (pero no se garantiza que lo altere).

**Polimorfismo.** La opción se muestra mediante una cadena de propiedad de la forma **isPolymorphic=true** (anulable) o bien **isPolymorphic=false** (no anulable). La ausencia de una opción explícita indica que la opción es verdadera, esto es, anulable.

**Alcance.** Las operaciones que tienen alcance de instancia se denotan dejando sin subrayar la cadena de operación. Una operación con alcance de clase se denota subrayando la cadena de nombre.

**Concurrencia.** La opción se muestra mediante una cadena de propiedad cuya forma es **concurrency=valor**, en donde el **valor** puede ser **sequential**, **guarded** o **concurrent**.

**Señales.** Si se quiere indicar que una clase admite una señal, se antepone la palabra reservada «**signal**» delante de la declaración de la operación, dentro de la lista de operaciones. Los

parámetros son los parámetros de la señal. La declaración puede no poseer un tipo de retorno. La respuesta del objeto a la recepción de la señal se muestra mediante una máquina de estados. Entre otras aplicaciones, esta notación puede mostrar la respuesta de los objetos de una máquina frente a las excepciones y situaciones de error, que deberían modelarse como señales.

### ***Opciones de representación***

La lista de argumentos y el tipo de retorno se pueden suprimir (simultáneamente, o por separado).

Una herramienta puede mostrar la indicación de visibilidad de una forma diferente, tal como emplear un icono especial o bien ordenar por grupos los elementos.

La sintaxis de la cadena de signature de la operación puede ser la de algún lenguaje de programación concreto, tal como C++ o SmallTalk. Se pueden incluir propiedades etiquetadas específicas en la cadena.

### ***Reglas de estilo***

- Típicamente, los nombres de las operaciones comienzan por una letra minúscula.
- Los nombres de las operaciones se muestran con letra normal.
- Las operaciones abstractas se muestran en cursiva.

## **Elementos estándar**

semantics.

## **operación abstracta**

Operación que carece de implementación, es decir, que tiene definida la especificación pero no el método. Es obligatorio que alguna subclase concreta proporcione su implementación.

*Véase abstracto/a, elemento generalizable, herencia, polimórfico/a.*

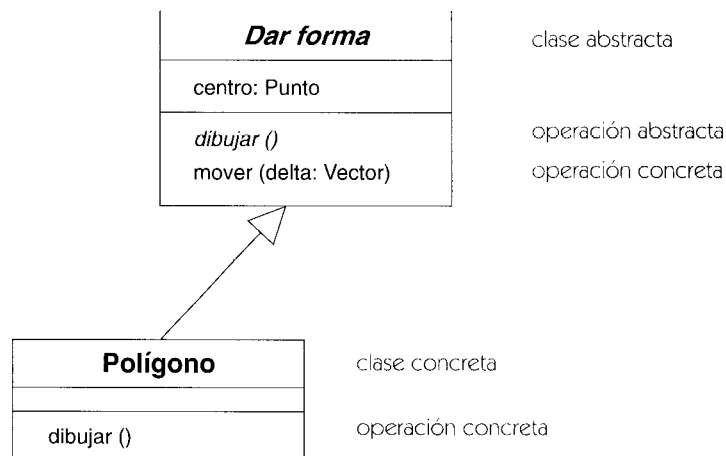
## **Semántica**

Si una operación se declara como abstracta en una clase, su implementación no aparece en dicha clase, y como consecuencia de ello la clase será necesariamente abstracta. Debe haber algún descendiente concreto que proporcione una implementación para la operación. Si una clase hereda una implementación de la operación pero declara dicha operación como abstracta, esta declaración invalida el método heredado por la clase. Si una operación se declara como concreta en una clase, la clase debe proporcionar una implementación (un método o una llamada) o heredarla de una clase antecesora. Si en una clase no se declara una operación, la clase hereda la declaración y la implementación de la operación (o su carencia de implementación) de sus clases antecesoras.

Una operación puede implementarse como un método o como una transición de una máquina de estados disparada por un evento de llamada. Cada clase debe declarar su propio método o evento de llamada para una operación o heredar una definición de una clase antecesora.

## Notación

El nombre de una operación abstracta debe aparecer en cursiva (ver Figura 13.151). También se puede utilizar la palabra **abstract** en la lista de propiedades que aparece después de la signatura de la operación.



**Figura 13.151** Clases y operaciones abstractas

## Discusión

La utilización más importante del concepto de herencia es permitir operaciones abstractas que puedan ser implementadas de forma distinta por cada clase descendiente concreta. Una operación abstracta permite invocar una operación sin saber de forma precisa cuál es la clase del objeto destino, siempre y cuando el objeto destino tenga la operación por el hecho de ser una instancia indirecta de una clase abstracta que tiene una declaración de la operación abstracta. El significado de declaraciones polimórficas de este tipo es determinar el tipo de objeto referido por el que invoca el mecanismo de herencia. El que realiza la llamada se libera no sólo de la molestia y el coste de escribir sentencias de código, sino que además no tiene por qué ser consciente de la existencia de posibles subclases de la clase abstracta, lo que significa que se pueden añadir más tarde subclases adicionales con nuevas implementaciones de la operación. De ese modo, las operaciones abstractas, el polimorfismo y la herencia facilitan la actualización de los sistemas mediante la adición de nuevos tipos de objetos y comportamientos sin necesidad de modificar el código que invoca el comportamiento genérico. Esto reduce significativamente el tiempo necesario para actualizar un sistema, y lo que es más importante, reduce la posibilidad de inconsistencias accidentales.

## ordenación

Cierta propiedad de un conjunto de valores, tal como el conjunto de valores relacionados con un objeto a través de una asociación, que indica si el conjunto está ordenado o desordenado.

*Véase también* asociación, extremo de asociación, multiplicidad.

## Semántica

Si el límite superior de multiplicidad del extremo de una asociación es mayor que uno, entonces hay un conjunto de objetos asociado a cada uno de los objetos del otro extremo de la asociación binaria. La propiedad de ordenación declara si el conjunto está ordenado o desordenado. Si está desordenado entonces los objetos del conjunto no tienen un orden explícito; forman un conjunto ordinario. Si está ordenado, los elementos del conjunto tienen un orden explícitamente impuesto. El orden de elementos forma parte de la información que representa la asociación, esto es, se trata de una información adicional que va más allá de la información contenida en los elementos en sí. Los elementos se pueden obtener por ese orden. Cuando se añade un nuevo enlace a la asociación, es preciso especificar su posición dentro de la secuencia; esto tiene que hacerlo la operación que lo añade. La posición puede ser un argumento de la operación o bien puede ser implícito. Por ejemplo, una operación dada puede situar un nuevo enlace al final de la lista de enlaces existentes, pero la localización del nuevo enlace debe ser especificada de algún modo.

Observe que un conjunto ordenado no es lo mismo que un conjunto cuyos elementos se hayan ordenado por uno o más de los atributos de los elementos. Un orden queda totalmente determinado por los valores de los elementos del conjunto. Por tanto, no añade información, aunque ciertamente puede resultar útil a efectos de acceder a los elementos. La información de una asociación ordenada, sin embargo, es adicional con respecto a la información de los elementos en sí.

La propiedad de ordenación es aplicable a cualquier elemento que admita una multiplicidad, tal como un atributo cuya multiplicidad sea mayor que uno.

Una relación ordenada se puede implementar de varias maneras, pero la implementación suele enunciarse en forma de una propiedad de generación de código especificada en algún lenguaje. Una extensión de implementación podría reemplazar la estructura de datos que contiene los elementos por la especificación genérica ordenada.

Un conjunto ordenado requiere una especificación por separado de la regla de ordenación en sí, que se dará en forma de restricción.

## Notación

La ordenación se especifica mediante una palabra clave entre llaves que se sitúa cerca del extremo de la ruta a que se aplica (Figura 13.152). La ausencia de una palabra reservada denota que no está ordenada. La palabra reservada **{ordered}** indica que se trata de un conjunto ordenado. A efectos de diseño, puede utilizarse la palabra reservada **{sorted}** para indicar un conjunto organizado mediante valores internos.



**Figura 13.152** Conjuntos ordenados y desordenados

Para aquellos atributos cuya multiplicidad sea mayor que uno, se puede situar una de las palabras reservadas de ordenación después de la cadena de atributos, entre llaves, formando parte de una cadena de propiedades.

Si se omite la palabra clave de ordenación, entonces el conjunto no está ordenado.

## Discusión

Un conjunto ordenado posee información en la ordenación, información que es adicional respecto a las entidades del conjunto en sí. Esto es una información real. Por tanto, no es derivable sino que se tiene que especificar cuándo se añade una entidad. En otras palabras, en toda operación que añada una entidad, es preciso especificar su posición dentro de la lista de entidades. Por supuesto, se puede implementar una operación de tal modo que la nueva entidad se inserte en alguna localización implícita, tal como el comienzo o el final de la lista. Y el hecho de que un conjunto esté ordenado no implica que se vaya a admitir cualquier ordenación de las entidades. Estas decisiones tiene que tomarlas quien haga el modelo. En general, la posición de la nueva entidad dentro de la lista es un parámetro de la operación de creación.

Observe que la ordenación de una asociación binaria tiene que ser especificada independientemente para cada dirección. La ordenación carece de sentido a no ser que la multiplicidad en una dirección sea mayor que uno. Una asociación puede ser completamente desordenada, ordenada en una dirección y no en la otra, o bien puede ser ordenada en ambas direcciones.

Consideremos una asociación entre las clases A y B que está ordenada en la dirección de B. Entonces lo normal será que los nuevos enlaces se añadan como una operación sobre un objeto A, especificando un objeto B y una posición en la lista de objetos B existentes para el nuevo enlace. Con cierta frecuencia, una operación sobre un objeto A crea un nuevo objeto B y también crea un enlace entre A y B. Es preciso añadir la lista a la lista de enlaces que mantiene A. Es posible crear un nuevo enlace desde el lado de B pero generalmente el nuevo enlace se insertará en una posición por defecto de la lista que va de A hasta B, porque la posición dentro de esa lista tiene escaso significado desde el lado de B. Por supuesto, un programador puede implementar situaciones más complicadas si fuera necesario.

Resulta poco frecuente hallar una asociación que esté ordenada en ambas direcciones, porque puede resultar engorroso especificar el punto de inserción en las dos direcciones. Pero es posible, especialmente si se añaden los nuevos enlaces en localizaciones por defecto en ambas direcciones.

Observe que un conjunto ordenado no contiene ninguna información adicional más allá de la información relativa al conjunto de entidades. La ordenación ahorra tiempo en un algoritmo pero no agrega información alguna. Se puede considerar como una optimización de diseño y no

es necesario incluirla en un modelo de análisis. Puede ser especificada como valor de la propiedad de ordenación, pero no requiere que una operación especifique una localización para la nueva entidad añadida al conjunto. La localización de la nueva entidad debe ser determinada automáticamente por el método que examina los atributos según los cuales se ordena la lista.

## padre

El elemento más general dentro de una relación de generalización. Para una clase recibe el nombre de superclase. Una cadena de una o más relaciones de padre (esto es, el cierre transitivo) es un antecesor. Lo contrario es un hijo.

*Véase generalización.*

## palabra clave

Se entiende que una palabra clave es un adorno textual que categoriza a un elemento del modelo cuando éste carece de su propia sintaxis distintiva.

*Véase también* marcador gráfico, estereotipo.

## Notación

Las palabras reservadas se emplean para aquellos elementos incorporados del modelo que carecen de su propia notación, así como para los estereotipos definibles por el usuario. La notación general para utilizar una palabra reservada consiste en encerrarla entre comillas («») «palabra clave».

Cuando la palabra clave forma parte de un símbolo de área, tal como un rectángulo de clase, entonces la palabra reservada se pone dentro de los límites del símbolo.

En el texto de este documento se describen algunas palabras claves que se tratan como palabras claves dentro de la notación. Hay otros nombres que están a disposición de los usuarios para que los empleen como nombres de estereotipo. No se permite utilizar un nombre de estereotipo que coincida con una palabra clave predefinida.

## Discusión

El número de símbolos visuales fácilmente distinguibles es limitado. Por tanto, la notación UML hace uso de palabras claves de texto para distinguir las variantes de un tema común, entre las que se encuentran las subclases de metamodelo de una clase base, los estereotipos de una clase base de metamodelo y los grupos de elementos de listas. Desde el punto de vista del usuario, la distinción de metamodelo entre subclases de metamodelo y estereotipos no suele ser importante, aunque lo sea, por supuesto, para los constructores de herramientas y para otras personas que implementen el metamodelo.

## paquete

Término que denota un mecanismo de propósito general para organizar en grupos los elementos. Los paquetes se pueden anidar dentro de otros paquetes. Un sistema puede corresponder a un único paquete de alto nivel, con todo el resto del modelo contenido recursivamente en él. En un paquete pueden aparecer tanto elementos del modelo como diagramas.

*Véase también* acceso, dependencia, importar, modelo, espacio de nombres, subsistema.

## Semántica

La palabra *paquete* alude a un agrupamiento de elementos y diagramas del modelo. Todo elemento del modelo que no forme parte de otro elemento del modelo deberá estar declarado únicamente en un espacio de nombres; el espacio de nombres que contiene la declaración de un elemento se denomina propietario del elemento. Un paquete es un espacio de nombres de propósito general que puede poseer cualquier clase de elemento de modelo que no esté limitado a una sola clase de propietario. Todo diagrama debe ser poseído exactamente por un paquete, que puede estar anidado dentro de otro paquete (y por tanto será poseído por este último). Los paquetes pueden contener paquetes subordinados y elementos ordinarios del modelo. Algunos paquetes pueden ser subsistemas o modelos. La descripción de todo el sistema puede concebirse como un único paquete de subsistema de alto nivel, que contiene absolutamente todo lo demás. Todas las clases de elementos de modelo de UML se pueden organizar en paquetes.

Los paquetes poseen elementos del modelo, subconjuntos del modelo y diagramas. Los paquetes son la base del control de configuraciones, del almacenamiento y del control de accesos. Todo elemento puede ser poseído directamente por otro elemento del modelo o por un único paquete, de tal modo que la jerarquía de posesión es un árbol estricto. Sin embargo, los elementos del modelo (incluyendo los paquetes) pueden hacer referencia a otros elementos de otros paquetes, así que la red de utilización es un grafo.

Las clases especiales de paquete son el modelo, el subsistema y el sistema. Un sistema denota un subsistema que es la raíz de la jerarquía del paquete. Se trata del único elemento del modelo que no es poseído por algún otro elemento del modelo. Indirectamente, incluye todo lo que hay en el modelo. Existen varios estereotipos predefinidos de modelo y de subsistema.

*Véase* el Capítulo 14, Elementos Estándar, para más detalles.

Los paquetes pueden tener relaciones de dependencia con otros paquetes. En la mayoría de los casos, estas dependencias no son otra cosa que un resumen de las dependencias que existen entre los contenidos de los paquetes. Un uso de dependencia entre dos paquetes significa que existe al menos una dependencia de uso entre elementos de los dos paquetes (esto no quiere decir que toda pareja de elementos tenga esa dependencia).

La dependencia de acceso es propia de los paquetes en sí y no es un compendio de dependencias de sus elementos. Indica que a los elementos del paquete cliente se les otorga permiso para tener relaciones con los elementos del paquete proveedor. Las relaciones también están sometidas a especificaciones de visibilidad. El acceso no significa que los nombres de los elementos que se encuentran en el paquete blanco ocupen el espacio de nombres del paquete origen: los espacios de nombres son diferentes y los elementos pueden identificarse de modo

único mediante nombres de ruta que incluyen a los paquetes anidados. La importancia de la variación de dependencia de acceso es como la sentencia **uses** de Ada. Añade los nombres del espacio de nombres del proveedor al espacio de nombres del cliente (no deben existir conflictos). Pero la dependencia de acceso no altera el espacio de nombres del cliente. Se trata, principalmente, de un mecanismo de control de acceso en los proyectos de desarrollo más extensos y no una relación semántica fundamental.

Un paquete anidado tiene acceso a todos los elementos que estén contenidos directamente en paquetes externos (hasta cualquier nivel de anidamiento) sin necesidad de dependencias de importación ni de visibilidades. Sin embargo, el paquete tiene que importar los paquetes que contenga para poder ver su interior. En general, un paquete contenido es un límite de encapsulamiento.

Los paquetes definen la visibilidad de los elementos que contienen como **privada**, **protegida** o **pública**. Los elementos privados no están disponibles de ningún modo fuera del paquete que los contiene (independientemente de las importaciones). Los elementos protegidos sólo están disponibles para aquellos paquetes que tengan generalizaciones al paquete que los contiene y los elementos públicos están disponibles para los paquetes que importan y para los descendientes del paquete.

Véase acceso para una descripción completa de las reglas de visibilidad que se aplican a los elementos de los distintos paquetes.

## Notación

Los paquetes se representan mediante un rectángulo grande con un rectángulo pequeño (una "ficha") asociado a una esquina (normalmente, el lado izquierdo de la parte superior del rectángulo grande). Se trata del icono de una carpeta.

Si no se muestra el contenido del paquete, entonces el nombre del paquete se sitúa dentro del rectángulo grande. Si se muestra el contenido del paquete, entonces el nombre del paquete se puede situar dentro de la ficha.

Se puede situar la cadena de una palabra reservada por encima del nombre del paquete. Entre las palabras reservadas se cuentan subsistema, sistema y modelo. Los estereotipos definidos por el usuario se anotan también con palabras reservadas pero no deben entrar en conflicto con las palabras reservadas predefinidas.

Se puede poner entre llaves una lista de propiedades detrás o por debajo del nombre del paquete. Ejemplo: {**abstract**}.

El contenido del paquete se puede mostrar dentro del rectángulo grande.

La visibilidad de un elemento de paquete fuera del paquete se puede indicar anteponiendo al nombre del elemento un símbolo de visibilidad ('+' denota pública, '-' significa privada y '#' quiere decir protegida).

Se pueden dibujar las relaciones entre símbolos de paquete para mostrar relaciones que existan al menos entre algunos de los elementos de los paquetes. En particular, la dependencia de otros paquetes (salvo las dependencias de permiso, tales como el acceso y la importación) implican que existe una o más dependencias entre los elementos).



## Opciones de representación

Las herramientas también pueden mostrar la visibilidad visualizando selectivamente aquellos elementos que satisfacen un determinado nivel de visibilidad; por ejemplo, mostrando únicamente los elementos públicos.

Las herramientas pueden mostrar la visibilidad empleando algún marcador gráfico, tal como un color o un tipo de letra.

## Reglas de estilo

Se supone que los paquetes que posean contenidos extensos se mostrarán como sencillo iconos con nombre, en los cuales se puede acceder dinámicamente al contenido “haciendo zoom” para llegar a una visualización detallada.

## Ejemplo

La Figura 13.153 muestra la estructura de paquete de un subsistema para el procesamiento de pedidos. El subsistema en sí se representa mediante un paquete con un estereotipo. Contiene varios paquetes ordinarios. Las dependencias entre paquetes se muestran mediante flechas discontinuas. La figura muestra también algunos paquetes externos de los cuales depende el subsistema. Éstos pueden ser componentes estandarizados o bien elementos de una biblioteca.

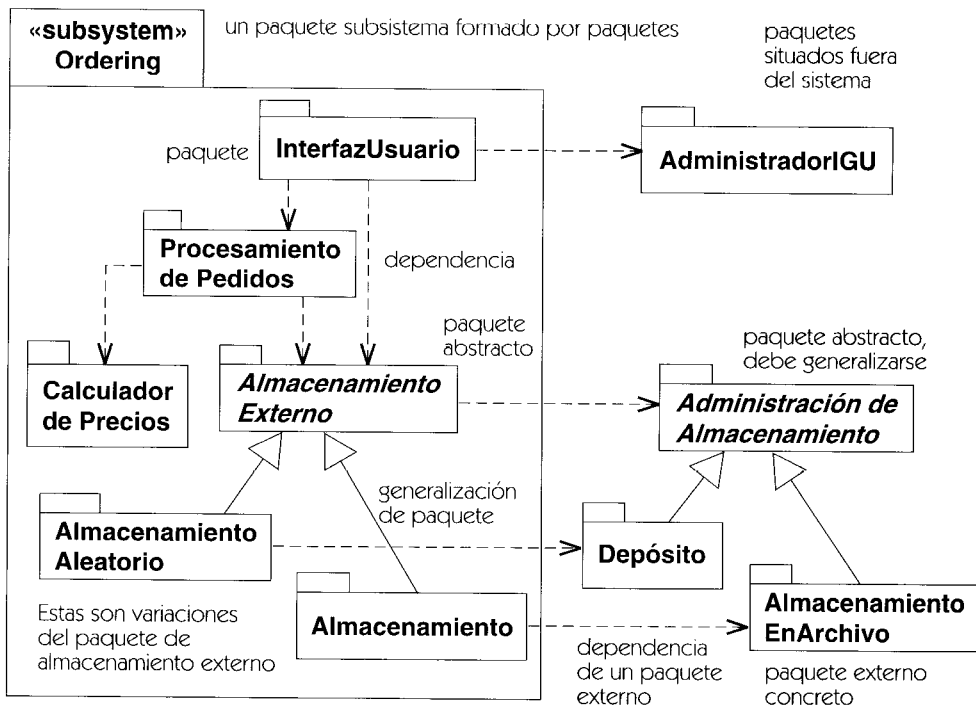


Figura 13.153 Los paquetes y sus relaciones

La generalización entre paquetes muestra las variaciones de un paquete genérico. Por ejemplo, el paquete **AlmacenamientoExterno** se puede implementar como un **AlmacenamientoAleatorio** o un **Almacenamiento**.

## Discusión

Los paquetes están destinados primordialmente a ser mecanismos de control de acceso y configuración que permitan a los desarrolladores, especialmente los que trabajan en grandes grupos, organizar grandes modelos y hacerlos evolucionar sin estorbarse unos a otros. Significan, inherentemente, lo que los desarrolladores quieran que signifiquen. Mas prácticamente, los paquetes deberían seguir algún tipo de límite semántico para que puedan ser útiles. Como están destinados a ser unidades de control de configuraciones, deberían contener elementos que tengan probabilidades de evolucionar al unísono. Los paquetes también sirven para agrupar aquellos elementos que deben ser compilados a la vez. Si un cambio de uno de los elementos impone la recompilación de otros elementos, entonces quizá convenga poner a ambos en un mismo paquete.

Todo elemento del modelo debe ser poseído exactamente por un paquete u otro elemento del modelo. En caso contrario, el mantenimiento del modelo, las versiones y el control de configuración se vuelven imposibles. El paquete que posee un elemento del modelo controla su definición. Se puede hacer referencia a él y es posible utilizarlo en otros paquetes, pero hacer un cambio en él requiere tener un permiso de acceso y el derecho de actualización para el paquete que lo posee.

## Elementos estándar

access, extend, facade, framework, stub, system.

## parámetro

Término que denota la especificación de una variable que se puede modificar, pasar o proporcionar. Los parámetros pueden tener nombre, tipo y dirección. Se utilizan en operaciones, mensajes, eventos y plantillas. Contrastar con: argumento.

Las dependencias de uso de parámetro relacionan a una operación que posee un parámetro o a una clase que posea esa operación con la clase del parámetro.

*Véase también* argumento, enlace.

## Semántica

Un parámetro es una reserva de espacio para un argumento, que se enlaza con él cuando se hace uso del elemento que lo contiene. Los parámetros restringen los valores que puede admitir el argumento. Tienen las partes siguientes:

|                   |                                                                                                                                                                                                                                    |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| valor por defecto | Es una expresión que proporciona el valor que se utilizará si no se proporciona un argumento para el parámetro. La expresión se evalúa cuando la lista de parámetros se enlaza con los argumentos.                                 |
| dirección         | Es la dirección del flujo de información del parámetro; se trata de una enumeración con los valores siguientes:                                                                                                                    |
| <b>in</b>         | Un parámetro de entrada pasado por valor. Los cambios efectuados en el parámetro no están disponibles para el emisor.                                                                                                              |
| <b>out</b>        | Un parámetro de salida. No existe valor de entrada. El valor final está disponible para el emisor.                                                                                                                                 |
| <b>inout</b>      | Un parámetro de entrada que se puede modificar. El resultado final está disponible para el emisor.                                                                                                                                 |
| <b>return</b>     | Valor proporcionado por una llamada. El valor está disponible para el emisor. Semánticamente, no tiene diferencias con respecto a un parámetro out, pero el resultado está disponible para su utilización en una expresión inline. |

Las opciones precedentes pueden no estar disponibles directamente en todos los lenguajes de programación, pero el concepto que subyace a todas ellas tiene sentido en la mayoría de los lenguajes y se puede hacer corresponder con una implementación razonable.

|        |                                                                                                                                                                                                                                                      |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| nombre | Es el nombre del parámetro. Tiene que ser único dentro de su lista de parámetros.                                                                                                                                                                    |
| tipo   | Se trata de una referencia de un clasificador (una clase, tipo de datos o interfaz en la mayoría de los procedimientos). El argumento que esté enlazado con el parámetro tiene que ser una instancia del clasificador o de uno de sus descendientes. |

## Notación

Todo parámetro se representa mediante una cadena de texto que se puede analizar sintácticamente para descomponerla en las distintas propiedades del parámetro. La sintaxis por defecto es:

dirección nombre : tipo valor-por-defecto

**Dirección.** La dirección se representa mediante una palabra clave que precede al nombre de la operación. Si la palabra clave está ausente, entonces la dirección es **in**. Las opciones son **in**, **out**, **inout** y **return**. Los parámetros **return** suelen mostrarse en otra posición distinta en la signatura de las operaciones, lugar en que no es necesario marcar su dirección.

**Nombre.** El nombre se representa mediante una cadena.

**Tipo.** El tipo se denota como una cadena que es el nombre de una clase, una interfaz o un tipo de datos.

**Valor por defecto.** El valor se representa mediante una cadena de expresión. El lenguaje de la expresión debería ser conocido por la herramienta (y debe ser especificable para la herramienta) pero no se muestra en formato canónico.

**Alcance.** Si el alcance es de clase, entonces se subraya la cadena de operación. Si el alcance es de instancia, entonces no se subraya la cadena de la operación.

**Dependencia paramétrica.** Una dependencia paramétrica se representa mediante una flecha discontinua que va desde la operación que tiene el parámetro o la clase que contiene a esa operación hasta la clase del parámetro; la flecha tiene asociado el estereotipo «**parameter**».

## **Ejemplo**

Matriz::transformación (in distancia: Vector, in ángulo: Real = 0): **return** Matriz

Se pueden omitir todas las etiquetas de dirección.

## **parámetro real**

Véase argumento.

## **participantes**

Denota la conexión de un elemento de un modelo con una relación o con una relación materializada. Por ejemplo, una clase participa en una asociación, un rol de clasificador participa en una colaboración.

## **patrón**

Término que denota una colaboración parametrizada que representa un conjunto de clasificadores, relaciones y comportamientos parametrizados, que se pueden aplicar a múltiples situaciones enlazando elementos del modelo (clases normalmente) con los roles del patrón. Es una plantilla de colaboración.

## **Semántica**

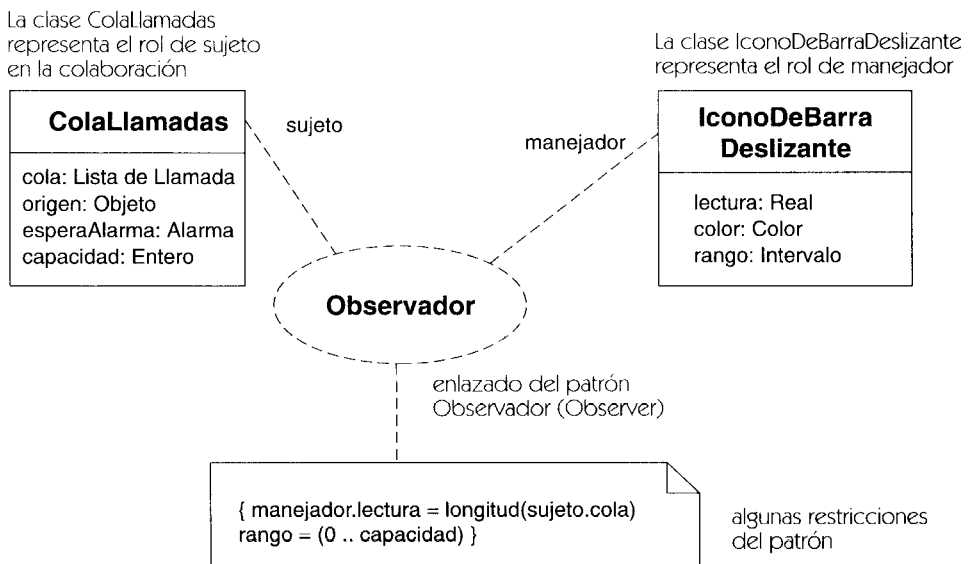
Los patrones representan una colaboración parametrizada que se puede emplear en múltiples ocasiones dentro de uno o más sistemas. Para ser un patrón, la colaboración tiene que ser utilizable en una amplia gama de situaciones, para justificar darle un nombre. Un patrón es una solución que demostradamente funciona en un cierto número de situaciones. No es necesariamente la única solución del problema, pero es una solución que ha sido efectiva en el pasado. Casi todos los patrones tienen ventajas y desventajas, que dependen de distintos aspectos del sistema global. El creador de modelos debe considerar estas ventajas y desventajas antes de tomar la decisión de utilizar un patrón.

Una colaboración parametrizada de UML representa las vistas estructural y de comportamiento de ciertas clases de patrones. Los patrones implican otros aspectos que no son modelados directamente por UML, tales como la lista de ventajas y desventajas de su utilización anterior. Hay muchos de estos aspectos que se pueden expresar en palabras. Véase [Gamma-95] para un tratamiento más completo de los patrones, así como un catálogo de patrones de diseño probados.

**Generar colaboraciones a partir de patrones.** Se puede utilizar una colaboración para especificar la implementación de estructuras de diseño. Es posible emplear la misma clase de colaboración en muchas ocasiones, parametrizando sus componentes. Un patrón es una colaboración parametrizada. En general, las clases de los roles que intervienen en la colaboración son parámetros. El patrón se instancia como una colaboración enlazando valores, normalmente clases, con sus parámetros. En el caso común de los roles parametrizados, la plantilla se enlaza especificando una clase para cada rol. Típicamente, los roles de asociación de un patrón no están parametrizados. Cuando se enlaza la plantilla, representan asociaciones implícitas entre las clases enlazadas con la colaboración, esto es, el enlazado de la plantilla para hacer una colaboración genera asociaciones adicionales.

## Notación

El enlazado de un patrón para producir una colaboración se representa mediante una elipse discontinua que contiene el nombre del patrón (Figura 13.154). Se traza una línea discontinua desde el símbolo de enlazado del patrón hasta las clases (u otros elementos del modelo) que participen en la colaboración. Todas las líneas se etiquetan con el nombre del parámetro. En la mayoría de los casos el nombre de un rol de la colaboración se puede emplear como nombre de parámetro. Por tanto, un símbolo de enlazado de patrón puede mostrar la utilización de un patrón de diseño, junto con las clases que realmente aparecen en esa utilización del patrón. El



**Figura 13.154** Enlazado de un patrón para formar una colaboración

enlazado del patrón no suele mostrar la estructura interna de la colaboración que se genera mediante el enlazado. Esto está implícito en el símbolo de enlazado.

## permiso

Denota una clase de dependencia que concede al cliente permiso para utilizar el contenido del elemento proveedor (sometido a las declaraciones de visibilidad de los elementos del mismo).

### Semántica

Los estereotipos de dependencia de permiso son `access`, `friend` e `import`. Nunca se utiliza un principio de dependencia al desnudo sin un estereotipo. Las dependencias de acceso y de importación se emplean en los paquetes. La dependencia de amistad se emplea con clases u operaciones como clientes y con clases como proveedores.

### Notación

Las dependencias de permiso se representan mediante una flecha discontinua que va del cliente (el elemento al que se da permiso) al proveedor (el elemento que concede el permiso) con la correspondiente palabra reservada de estereotipo asociada a la flecha.

### Elementos estándar

`friend`, `import`.

## plantilla

Elemento parametrizado de un modelo. Para utilizarla, los parámetros tienen que estar asociados (en el tiempo de modelado) con valores reales. Sinónimo: elemento parametrizado.

*Véase también* enlace, elemento ligado.

### Semántica

Una plantilla es el descriptor de un elemento con uno o más parámetros formales. Por tanto, define una familia de elementos potenciales, estando especificado cada uno de los elementos por el enlazado de los parámetros con valores reales. Típicamente, los parámetros son clasificadores que representan tipos de atributos, pero también pueden representar enteros o incluso operaciones. Los elementos subordinados dentro de la plantilla se definen en términos de los parámetros formales, así que también quedan enlazados cuando se enlaza la plantilla en sí como los valores reales.

Una plantilla de clase es el descriptor de una clase parametrizada. El cuerpo de una plantilla puede contener muchas apariciones de los parámetros formales, así como un elemento por defecto que representa la plantilla en sí. Las clases reales se producen enlazando los parámetros con valores. Los atributos y operaciones existentes dentro de la clase plantilla se pueden definir en términos de los parámetros formales. La clase plantilla también puede tener relaciones, tales como asociaciones y generalización, entre ella y uno de sus parámetros. Cuando se enlaza la plantilla, el resultado es una relación entre la clase de plantilla enlazada y las clases enlazadas con los parámetros de la relación.

Las clases de plantilla no son clases utilizables directamente porque poseen parámetros que no están enlazados. Es preciso enlazar los parámetros con valores reales para crear una clase real. Sólo una clase real podrá ser padre o destino de una asociación (una asociación monodireccional procedente de la plantilla y que llega a otra clase es admisible, sin embargo). Una clase de plantilla puede ser una subclase de una clase ordinaria, lo cual implica que todas las clases formadas al enlazar la plantilla son subclases de la clase dada. También puede ser hija de uno de los parámetros de plantilla; esto implica que la clase de plantilla enlazada es hija de la clase pasada como argumento.

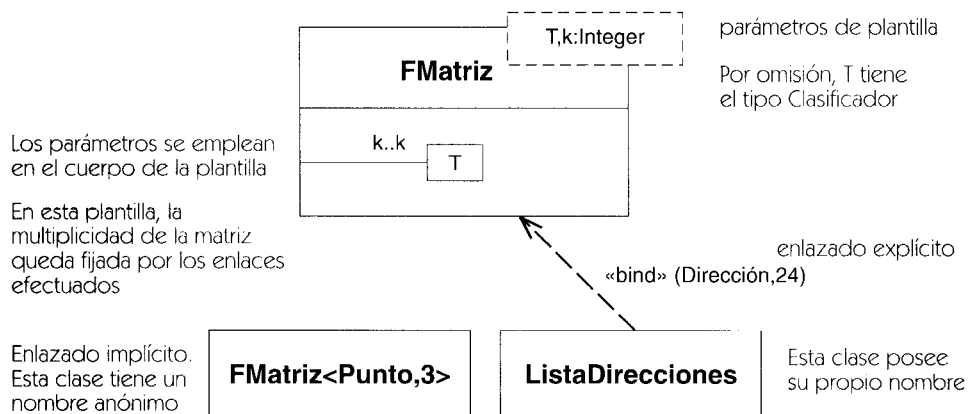
Es posible aplicar la parametrización a otros elementos del modelo, tales como las colaboraciones e incluso paquetes completos. La descripción que se da aquí para las clases se aplica en la forma evidente a los otros elementos de modelado.

El contenido de una plantilla no está sometido directamente a las reglas de buena formación que afectan a los modelos. Esto se debe a que incluyen parámetros que no tienen una semántica completa mientras no hayan sido enlazados. Una plantilla es una especie de elemento de modelo de segundo nivel, no un elemento que modele directamente los sistemas sino uno que modela otros elementos del modelo. Por tanto, el contenido de la plantilla está fuera de la semántica del sistema. Los resultados de enlazar una plantilla son elementos ordinarios del modelo, sometidos a las reglas de buena formación y son elementos normales del sistema blanco. Se podrían derivar ciertas reglas de buena formación para plantillas a partir de las consideraciones de que los resultados enlazados tienen que ser bien formados, pero no vamos a intentar enumerarlas. En cierto sentido, cuando se enlaza una plantilla su contenido se duplica y los parámetros son sustituidos por los argumentos. El resultado pasa a formar parte del modelo efectivo como si se hubiese incluido directamente.

Hay otras clases de clasificadores, como los casos de uso y las señales, que también se pueden parametrizar. Las colaboraciones también se pueden parametrizar; entonces son patrones.

## Notación

Se superpone un pequeño rectángulo discontinuo en la esquina superior derecha del rectángulo de clase u otro elemento de modelado. El rectángulo discontinuo contiene una lista de parámetros formales para la clase. Cada parámetro tiene su nombre y un clasificador. La lista no puede estar vacía (si lo estuviera, no sería una plantilla) aunque se puede suprimir en la representación. El nombre, los atributos y las operaciones de la clase parametrizada aparecen normalmente en el rectángulo de clase, pero también se pueden incluir apariciones de los parámetros formales. Las otras clases de elementos parametrizados se tratan análogamente. Las apariciones de los parámetros formales pueden producirse dentro del cuerpo de la



**Figura 13.155** Notación de plantilla empleando un parámetro como referencia

plantilla, por ejemplo, para mostrar una clase relacionada que es identificada por uno de los parámetros.

Los parámetros tienen la sintaxis:

nombre :tipo

donde nombre es un identificador del parámetro, con alcance de plantilla; y

donde tipo es una cadena que denota una expresión de tipo para el parámetro.

Si se omite el nombre del tipo, se supone que se trata de una expresión de tipo que al resolverse producirá un clasificador, tal como un nombre de clase o un tipo de datos. Los demás tipos de parámetros (tales como **Entero**) deben mostrarse explícitamente y tienen que producir expresiones de tipo válidas cuando se resuelven.

La Figura 13.155 muestra una plantilla con un parámetro entero y un parámetro de clase. La plantilla tienen una asociación con uno de sus parámetros.

## Discusión

El *modelo efectivo* es el modelo implícito que resulta de enlazar todas las plantillas; se trata del modelo implícito que describe el sistema. Los parámetros de plantilla no tienen significado dentro del modelo efectivo en sí, porque ya se habrán enlazado. Sólo se pueden utilizar dentro del alcance del cuerpo de la plantilla en sí. Esto basta para manejar los elementos constitutivos contenidos en el elemento parametrizado, por ejemplo, para atributos o operaciones contenidos en una clase parametrizada.

Los elementos que normalmente son externos respecto al elemento parametrizado plantean más dificultades. Por ejemplo, una clase puede tener asociación o generalizaciones que lleguen a otras clases. Si esas clases son parámetros de la plantilla, no puede formar parte del modelo efectivo, y sin embargo tampoco forman parte de una clase ordinaria. Por tanto, un elemento parametrizado contiene un *cuerpo* que representa un fragmento del modelo. El fragmento del modelo no forma parte de la plantilla en sí y puede incluir parámetros de plantilla tales como un



parámetro que represente una clase (que todavía no está especificada). Cuando se enlaza la plantilla, el cuerpo se copia implícitamente; los parámetros son reemplazados por los argumentos y la copia pasa a formar parte del modelo efectivo. Cada instanciación de la plantilla produce una adición al modelo efectivo. La Figura 13.156 muestra un ejemplo.

El cuerpo de la plantilla contiene implícitamente un elemento que representa el elemento de plantilla instanciado en sí; por ejemplo, la clase producida al enlazar una plantilla. Este elemento implícito se puede emplear para relaciones de construcción tales como asociaciones y generalizaciones con parámetros de plantilla. En la notación, los parámetros se dibujan dentro de los límites de la plantilla y hay una conexión al interior de los límites de la plantilla que denota una relación con el elemento instanciado implícito de la plantilla. Cuando se instancia la plantilla, éstas se transforman en relaciones del modelo efectivo entre el elemento de plantilla enlazado (recién instanciado) y los elementos (previamente existentes) que son argumentos de la plantilla.

Una plantilla puede ser hija de otro elemento. Esto significa que todo elemento enlazado generado a partir de ella es hijo del elemento dado. Por ejemplo, en la Figura 13.156, toda variable del tipo matricial (**VMatriz**) es una matriz (**Matriz**). Por tanto, **VMatriz<Punto>** es hijo de **Matriz**, **VMatriz<Dirección>** es hijo de **Matriz** y así sucesivamente.

Normalmente una plantilla no puede ser padre de otro elemento. Esto significaría que todo elemento generado al enlazar el padre sería padre del otro elemento. Aunque quizá alguien pudiera atribuir un significado a semejante situación, es poco probable.

Dos plantillas no pueden tener asociaciones entre sí, simplemente porque comparten el mismo nombre de parámetro. (Intentar hacer esto significaría que toda instanciación de la primera plantilla estaría relacionada con toda instanciación de la segunda plantilla, que no es lo que suele desearse normalmente. Esta cuestión ha sido malinterpretada por algunos autores en el pasado.) Los parámetros sólo tienen alcance dentro de su propia plantilla. Utilizar el mismo nombre de parámetro en dos plantillas no hace que sea el mismo parámetro. En general, si dos plantillas tienen elementos parametrizados que deben estar relacionados, entonces una de las plantillas debe instanciarse dentro del cuerpo de la otra. (Recuerde que una plantilla es instan-

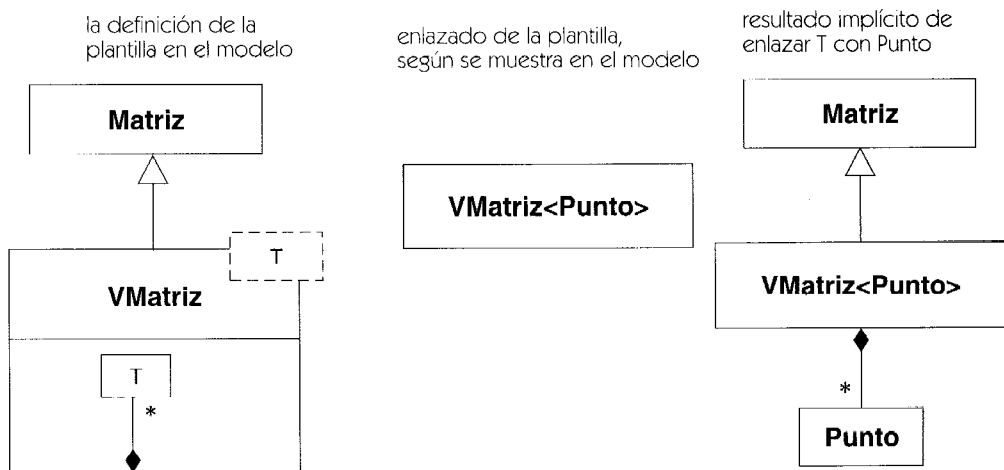
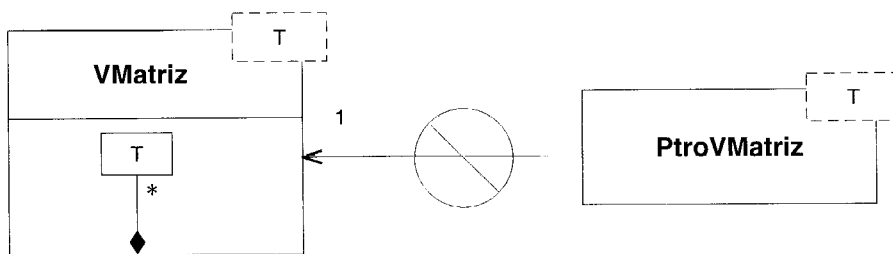


Figura 13.156 Plantilla con relación con uno de sus parámetros

¡MAL! Esto no tiene sentido. Las T no están relacionadas porque se encuentran en alcances distintos.



¡BIEN! Es preciso instanciar una plantilla para construir una asociación.

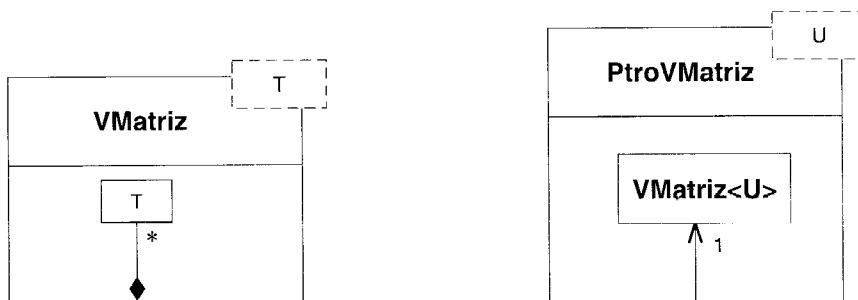


Figura 13.157 Asociaciones entre plantillas

ciada implícitamente dentro de su propio cuerpo. Por tanto, ambas plantillas son instanciadas efectivamente dentro del cuerpo y consiguientemente las relaciones son entre los elementos instanciados.) La Figura 13.157 muestra un intento incorrecto y otro correcto de definir semejante relación; en este caso, se ha empleado una clase parametrizada «**puntero**» que apunta a una matriz parametrizada de igual tipo.

Se puede utilizar un enfoque similar para declarar una clase parametrizada que sea hija de otra clase de plantilla enlazada con el mismo parámetro. El parámetro se utiliza para enlazar una copia de cada una de las otras plantillas. Entonces puede construirse una asociación entre las copias instanciadas de las plantillas. En la mayoría de los casos prácticos, no es necesario hacer esto porque se puede declarar la relación en el cuerpo de alguna de las plantillas.

## polimórfico/a<sup>5</sup>

Indica una operación cuya implementación (un método o una máquina de estados disparada por un evento de llamada) puede ser proporcionada por una clase descendiente. Una operación que no sea polimórfica es una operación hoja.

*Véase también* operación abstracta, generalización, herencia, método.

<sup>5</sup> Utilizaremos indistintamente los términos polimorfo y polimórfico para traducir el término inglés “polymorphic”. *N. del Revisor.*

## Semántica

Si una operación es polimórfica, entonces se le puede proporcionar un método en una clase descendiente (tanto si ya se ha suministrado un método en la clase original como si no). En caso contrario, es preciso que esté disponible un método para esa operación en la clase en que se declara la operación y el método no podrá ser anulado en una clase descendiente. Se dice que un método está disponible cuando lo declara una clase o cuando es heredado de un antecesor. Una operación abstracta tiene que ser polimórfica (porque no tiene implementación directa). Una operación no es polimórfica si se declara como operación hoja.

Si una operación se declara polimorfa en una clase, esto es, si no se declara como hoja, entonces puede ser declarada como hoja en alguna clase descendiente. Esto evita que sea redefinida o anulada en algún descendiente posterior. Las operaciones hoja no se pueden declarar polimórficas en clases descendientes. No se podrán anular ni redefinir independientemente de la profundidad.

UML no impone unas reglas de combinación de métodos si se declara un método en una clase y se redefine este método en una clase descendiente (*véase* la discusión siguiente). Los mecanismos, como pueda ser la declaración de métodos antes, después y lateralmente, se pueden manejar tal como convenga para un lenguaje específico empleando valores etiquetados. Las acciones tales como llamar explícitamente al método heredado son, por supuesto, dependientes del lenguaje de acción, en todo caso.

## Notación

Se declara una operación no polimorfa empleando la palabra reservada **{leaf}** (*hoja*). En caso contrario, se supone que es polimorfa.

## Discusión

Toda operación abstracta es necesariamente polimórfica. En caso contrario, no se podría implementar en modo alguno. Bertrand Meyer las llama operaciones diferidas porque su especificación se define en una clase pero su implementación se retrasa hasta las subclases. Se trata de una utilización esencial de la herencia, quizá la más esencial de todas tanto para el modelado como para la programación. Mediante el uso de la herencia, se pueden aplicar parámetros a conjuntos de objetos de clases mixtas. El emisor no necesita conocer o determinar la clase a que pertenece cada objeto. Sólo es necesario que todos los objetos se adapten a una clase antecesora que define las operaciones deseadas. La clase antecesora no necesita definir las operaciones. Basta con que se limite a definir sus firmas. El emisor ni siquiera necesita conocer la lista de posibles subclases. Esto significa que se pueden añadir posteriormente nuevas subclases, sin perturbar las operaciones polimórficas que les son aplicables. El código fuente que invoca a las operaciones no necesita modificaciones cuando se añaden nuevas subclases. La posibilidad de añadir nuevas clases después de haber escrito el código original es uno de los pilares fundamentales de la tecnología orientada a objetos.

Una aplicación más problemática del polimorfismo es la sustitución (también la anulación puede dar problemas) de un método definido en una clase por un método distinto definido en una subclase. Suele decirse que esto es una forma de compartir, pero es peligroso. La redefinición no es incremental, así que todo lo que haya en el método original tiene que reproducirse en el método hijo, aunque sólo sea para efectuar un pequeño cambio. Esta clase de repetición es

proclive a errores. En particular, si el método original se cambia posteriormente, no hay garantía de que el método hijo sea modificado también. Hay ocasiones en que una subclase puede hacer uso de una implementación completamente distinta de cierta operación, pero hay muchos expertos que desaconsejan esta redefinición por el peligro que implica. En general, los métodos deberían ser bien completamente heredados o bien diferidos; en este último caso no hay implementación en la superclase así que no hay peligro de redundancia ni de inconsistencia.

Para permitir que una subclase extienda la implementación de una operación sin perder el método heredado, la mayoría de los lenguajes de programación ofrece alguna forma de combinación de métodos que hace uso del método heredado pero permite añadir código adicional. En C++ un método heredado puede ser invocado explícitamente por el nombre de la clase y de la operación, lo cual inserta rígidamente la jerarquía de clases en el código: no es un enfoque muy orientado a objetos. En SmallTalk, un método puede invocar a una operación de **super**, lo cual da lugar a que la operación sea manejada por el método heredado. Si cambia la jerarquía de clases, entonces la herencia sigue funcionando, posiblemente con un método de otra clase diferente. Sin embargo, el método que anula debe proporcionar explícitamente una llamada a **super**. Los errores se producen, porque los programadores se olvidan de insertar las llamadas cuando se produce un cambio. Por último, CLOS ofrece unas reglas muy generales y complicadas para la combinación automática de métodos que puede invocar a varios métodos durante la ejecución de una sola operación de llamada. La operación global está implementada a partir de varios fragmentos en lugar de estar limitada a un único método. Esto es muy general pero resulta más difícil de manejar para el usuario.

UML no impone un único enfoque de combinación de métodos. La combinación de métodos es una cuestión de variación semántica. Se puede emplear cualquiera de estos enfoques. Si el lenguaje de programación no es muy potente en lo tocante a la combinación de métodos, entonces es posible que una herramienta de modelado pueda servir de ayuda para generar el código adecuado en ese lenguaje de programación, o quizá pueda emitir advertencias relativas a posibles errores causados por inacción si se emplea la anulación de métodos.

## postcondición

Es una restricción que tiene que satisfacerse cuando se produzca la terminación de una operación.

### Semántica

Una postcondición es una expresión booleana que tiene que ser verdadera una vez finalizada la ejecución de una operación. Se trata de una aserción<sup>6</sup>, no es una sentencia ejecutable. Dependiendo de la forma exacta de la expresión, quizá sea posible verificarla automáticamente por anticipado. Puede ser útil verificar la postcondición después de la operación, pero esto entra dentro de los límites de la depuración de programas. Se supone que la condición es verdadera y cualquier otra cosa es un error de programación. Una postcondición es una restricción que afecta a quien implementa una operación. Si no se satisface, entonces la operación se ha implementado de forma incorrecta.

*Véase también* invariante, precondition.

<sup>6</sup> El término "assertion" también puede ser traducido como "aserto" o "aseveración", aunque nosotros hemos preferido el más común de "aserción". *N. del Revisor.*

## Estructura

Las postcondiciones se modelan como las restricciones, con el estereotipo «**postcondition**», que está asociado a una operación.

## Notación

Se puede mostrar una postcondición en una nota con la palabra reservada «**postcondition**». La nota está asociada a la operación afectada.

## Ejemplo

La Figura 13.158 muestra una postcondición aplicada a una operación de clasificación de una matriz. El nuevo valor de la matriz (a') está relacionado con el valor original (a). Este ejemplo se expresa en lenguaje natural estructurado. También se podría hacer la especificación en un lenguaje más formal.

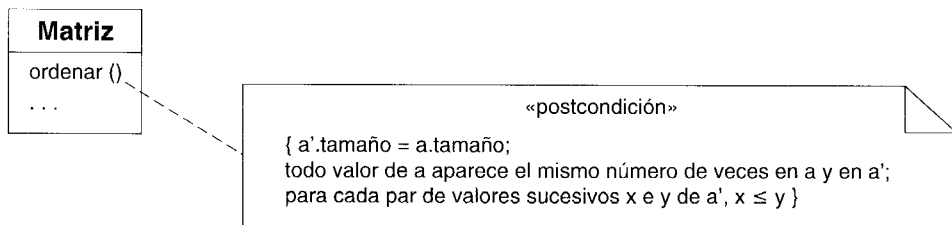


Figura 13.158 Postcondición

## precondición

Una restricción que tiene que ser cierta cuando se invoca a una operación.

## Semántica

Se trata de una expresión booleana que tiene que ser verdadera cuando se llama a una operación. Es responsabilidad del emisor satisfacer esa condición. No se trata de una condición que tenga que verificar el receptor. Una precondición no es una condición de guarda; es una condición que tiene que *ser* cierta y no una forma de ejecutar opcionalmente una operación. Puede resultar útil verificar la precondición al principio de la operación para más fiabilidad, pero esto entra en los límites de la depuración de programas. Se supone que la condición es verdadera y cualquier otra circunstancia es un error de programación. Si no se cumple la condición, no se puede hacer afirmación alguna con respecto a la integridad de la operación o del sistema. Tiene probabilidades de ser un fracaso absoluto. En la práctica, al verificar explícitamente las precondiciones en el receptor se pueden detectar muchos errores.

*Véase también* invariante, postcondición.

## Estructura

Las precondiciones se modelan como restricciones con el estereotipo «**precondition**» que se asocia a una operación.

## Notación

Se puede mostrar una precondición en una nota con la palabra reservada «**precondition**». La nota se asocia a la operación afectada.

## Ejemplo

La Figura 13.159 muestra una precondición aplicada a un operador de producto matricial.

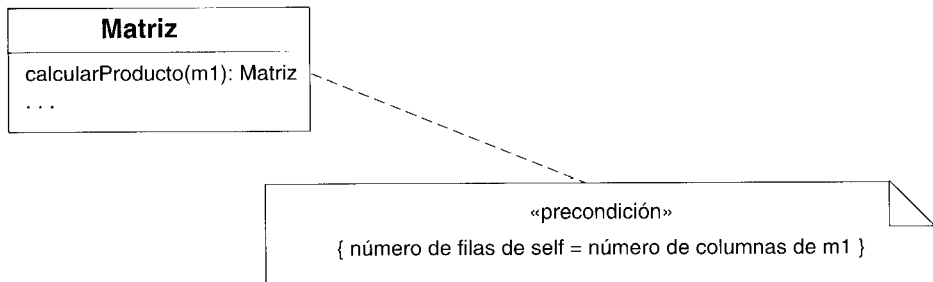


Figura 13.159 Precondición

## principio de capacidad de sustitución

Principio consistente en que dada la declaración de una variable o parámetro cuyo tipo declarado es X, toda instancia de elemento que sea descendiente de X puede utilizarse en lugar del valor real sin violar la semántica de la declaración ni de su utilización. En otras palabras, se puede reemplazar una instancia de un elemento antecesor por una instancia de un elemento descendiente. (Atribuido a Barbara Liskov.)

*Véase también* generalización, herencia de implementación, herencia, herencia de interfaz, polimórfico/a, herencia privada.

## Discusión

El propósito es asegurar que las operaciones polimórficas funcionen libremente. No se trata de un principio lógico sino más bien de una regla práctica de programación que proporciona un cierto grado de encapsulamiento. La relación de generalización apoya al principio de capacidad de sustitución.

La consecuencia del principio de capacidad de sustitución es que un hijo no puede eliminar ni redefinir propiedades de su padre. En caso contrario, no se podría utilizar al hijo como sustituto en una situación en que se hubiera declarado la utilización del padre.

## privado

Se trata de un valor de visibilidad que indica que el elemento dado no es visible fuera de su propio espacio de nombres, ni siquiera para los descendientes de ese espacio de nombres.

## proceso de desarrollo

Un conjunto de pautas y un conjunto de actividades de trabajo parcialmente ordenadas pensadas para producir software de una manera controlada, reproducible. El propósito de un proceso de desarrollo de software es asegurar el éxito y la calidad de un sistema acabado.

*Véase también* etapas de modelado.

## Discusión

UML es un lenguaje de modelado, no un proceso, y su propósito es describir los modelos que se pueden producir por varios procesos de desarrollo. Para la estandarización, es más importante describir los artefactos resultantes de un desarrollo que el proceso de producirlos. Eso es porque hay muchas buenas formas de construir un modelo, y un modelo acabado puede ser utilizado sin saber cómo fue producido. Sin embargo, UML está pensado para apoyar una amplia gama de procesos.

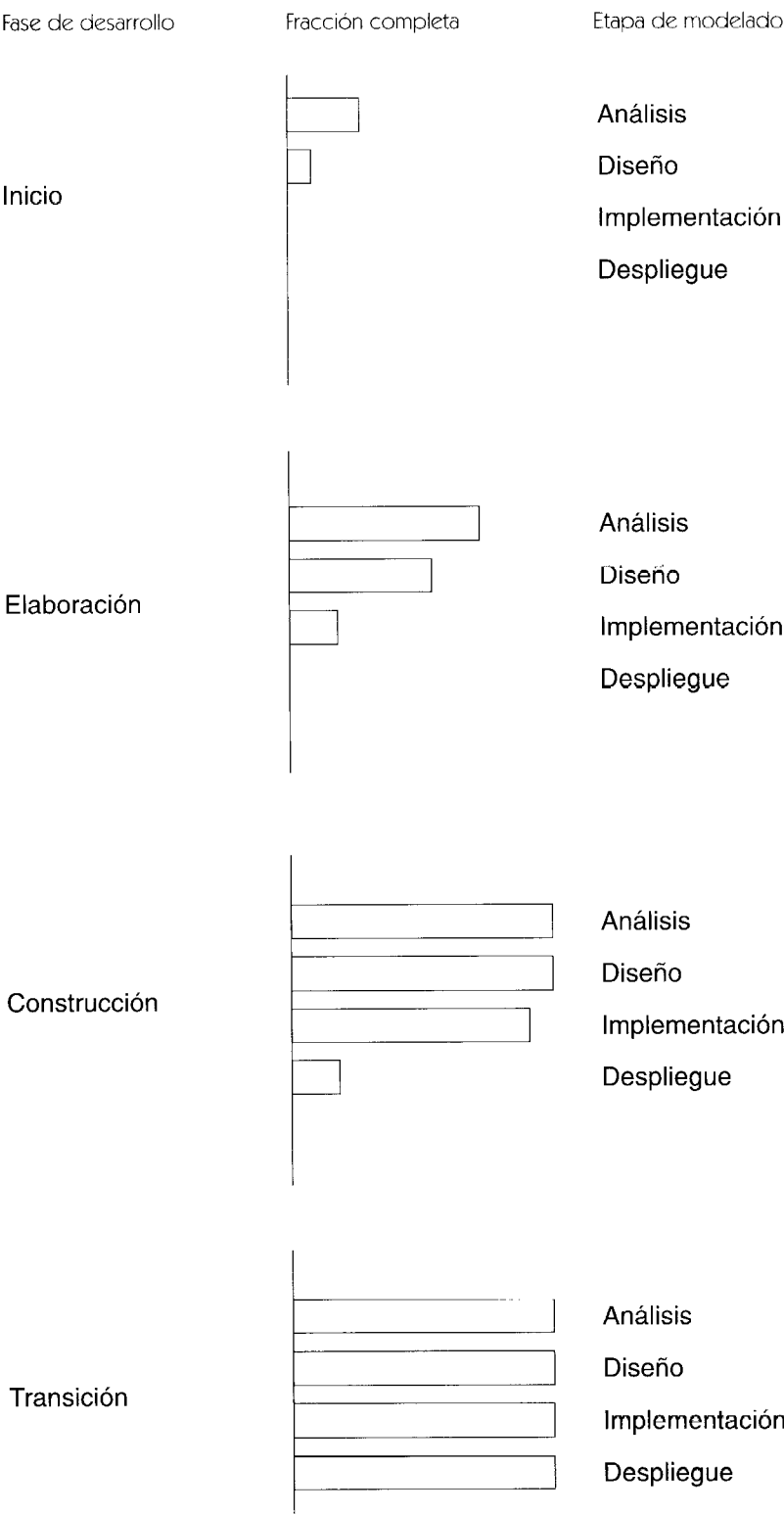
Para más detalles del proceso iterativo, incremental, guiado por casos de uso, centrado en la arquitectura que promueven los autores de este libro. *Véase* [Jacobson-99].

### ***La relación de las etapas de modelado y las fases de desarrollo***

Las etapas de modelado encajan dentro de un proceso de desarrollo iterativo, que tiene las fases de inicio, elaboración, construcción, y transición (fase). Las fases son secuenciales dentro de una entrega de una aplicación, pero cada fase incluye una o más iteraciones. Dentro de una iteración, los elementos individuales del modelo se mueven a lo largo del camino desde el análisis hacia el despliegue, cada uno a su propio ritmo. Aunque las fases del desarrollo y las etapas de modelado no se sincronizan, hay una correlación.

En las fases anteriores del desarrollo y las iteraciones anteriores de una fase, hay más énfasis en las etapas de modelado anteriores.

La Figura 13.160 muestra el equilibrio del esfuerzo durante las fases e iteraciones sucesivas. Durante la inicio, el foco de interés está principalmente en análisis, con un esqueleto de los elementos que progresa hacia el diseño y la implementación durante la elaboración. Durante la construcción y la transición, todos los elementos deben, eventualmente, llegar a su terminación.



**Figura 13.160** Progreso después de cada fase de desarrollo



## proceso/procesar

1. Unidad compleja de concurrencia y ejecución en un sistema operativo. Véase hilo, que incluye los procesos complejos y ligeros. Si es necesario, se puede hacer una distinción de implementación empleando estereotipos.
2. Un proceso de desarrollo de software; son los pasos y directrices mediante los cuales se desarrolla un sistema.
3. Ejecutar un algoritmo o en general manejar algo dinámicamente.

## producto

Dícese de los resultados del desarrollo, tales como modelos, código, documentación y planes de trabajo; también puede denotar un producto de trabajo.

## propiedades

Término genérico que denota un valor con nombre que aporta información acerca de un elemento del modelo. Las propiedades tienen impacto semántico. Ciertas propiedades están definidas en UML, otras pueden ser definidas por el usuario.

Véase atributo, relación, valor etiquetado.

## Semántica

Entre las propiedades se cuentan los atributos incorporados (definidos por UML) así como los valores etiquetados (definidos por el usuario) y las relaciones (definidas por el usuario) asociadas a un elemento. Desde el punto de vista de un usuario, suele carecer de importancia el que una propiedad sea incorporada o esté implementada como un valor etiquetado.

## Discusión

Obsérvese que se utiliza la palabra *propiedad* en un sentido general, para denotar cualquier valor que esté asociado a un elemento del modelo, incluyendo los atributos, asociación y valores etiquetados. En este sentido, puede incluir los valores alcanzables indirectamente que se pueden hallar comenzando en un determinado elemento.

## protegida

Dícese de aquel valor de visibilidad que indica que el elemento dado sólo es visible fuera de su propio espacio de nombres para los descendientes del espacio de nombres.

## proveedor

Elemento que proporciona servicios que pueden ser invocados por otros. Contrastar con: cliente. En la notación, el proveedor aparece en la punta de flecha de una flecha discontinua de dependencia.

*Véase dependencia.*

## proyección

Correspondencia entre un conjunto y un subconjunto del mismo. La mayoría de los modelos y de los diagramas son proyecciones del conjunto completo de información que está potencialmente disponible.

## pseudoatributo

Denota un valor relacionado con una clase que se comporta como un atributo, a saber, que tiene un valor exclusivo para cada instancia.

*Véase también discriminador, nombre de rol.*

## Semántica

Entre los pseudoatributos se cuentan los nombres de roles de asociación y los discriminadores de generalización. Un nombre de rol de asociación es un pseudoatributo de la clase que se encuentra al otro extremo de la asociación. Un discriminador de generalización es un pseudoatributo del elemento padre. En todos los elementos hijos, el valor del discriminador es el nombre del elemento hijo.

Se puede utilizar un pseudoatributo como nombre en una expresión para recuperar un valor de un objeto. Dado que los nombres de atributos y los nombres de pseudoatributos se pueden emplear en las expresiones, se encuentran en el mismo espacio de nombres y tienen que ser únicos en ese espacio de nombres. Los nombres también tienen que ser únicos respecto a los nombres de atributos y pseudoatributos heredados.

## pseudoestado

Denota un vértice de una máquina de estados que posee la forma de un estado pero no se comporta como un estado completo. Véase estado abreviado externo, estado de historia, estado de unión o conjunción, estado inicial.

## Semántica

Cuando un pseudoestado está activado, hay una máquina de estados que no ha finalizado su paso que se ejecuta hasta finalizar y que no procesa eventos. Entre los pseudoestados se cuentan el es-

tado inicial, el estado de unión o conjunción, el estado abreviado externo y el estado de historia. Los pseudoestados se utilizan para encadenar segmentos y una transición a uno de ellos implica otra transición automática adicional a otro estado sin necesidad de que intervenga un evento.

Los estados finales y de sincronización no son pseudoestados. Son estados especiales que pueden permanecer activos cuando una máquina de estados ya ha finalizado su paso que se ejecuta hasta finalizar, pero tienen restricciones relativas a las transiciones que pueden partir de ellos.

## pública

Dícese de aquel valor de visibilidad que indica que el elemento dado es visible fuera de su propio espacio de nombres.

## punto de extensión

Un marcador con nombre que se refiere a una localización o a un conjunto de localizaciones dentro de la secuencia de comportamiento para un caso de uso, en el cual se puede insertar comportamiento adicional. Una declaración de punto de extensión abre la posibilidad de extensión al caso de uso. Un segmento de inserción es una secuencia de comportamiento en un caso de uso de extensión (un caso de uso relacionado con un caso de uso base por una relación de extensión). La relación de extensión contiene una lista de nombres de puntos de extensión que indican el lugar donde los segmentos de la inserción del caso de uso extendido insertan su comportamiento.

*Véase también* extender, caso de uso.

## Semántica

Un punto de extensión tiene un nombre y se refiere a un conjunto de una o más localizaciones dentro de una secuencia de comportamiento de un caso de uso. Un punto de extensión puede referirse a una sola localización entre dos pasos del comportamiento dentro de un caso de uso. Además, puede referirse a un conjunto de localizaciones discretas. Puede también referirse a una región dentro de una secuencia de comportamiento (éste no es nada más que el conjunto de todas las localizaciones entre los pasos de la secuencia).

Una localización es un estado dentro de la máquina de estados que describe un caso de uso, o el equivalente de una descripción diferente —entre dos sentencias de una lista de sentencias o entre dos mensajes en una interacción.

Una relación de extensión contiene una condición opcional y una lista de referencias a puntos de extensión iguales en número al número de los segmentos de inserción en el caso de uso extendido. Un segmento de inserción puede ser realizado si la condición está satisfecha mientras que una instancia del caso de uso está ejecutando el caso de uso base en cualquier localización en el punto de extensión que corresponde al segmento de la inserción.

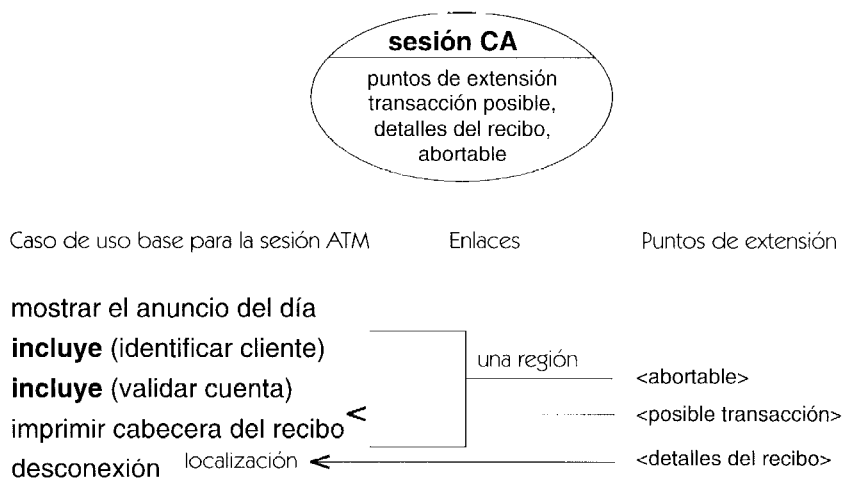
La localización de un punto de extensión puede ser cambiada sin afectar su identidad. El uso de los puntos de extensión con nombre separa la especificación de las secuencias de los detalles internos del caso de uso base. El caso de uso base puede ser modificado o ser cambiado sin afectar las extensiones. Por otra parte, un punto de la extensión se puede mover dentro de la base sin afectar la relación o el caso de uso extendido. Como con todos los tipos de modularidad, esta independencia requiere una buena elección de los puntos de extensión y no está garantizada bajo todas las circunstancias.

## Notación

Los puntos de extensión para un caso de uso se pueden enumerar en un compartimento llamado **puntos de extensión**.

Los puntos de extensión deben también referirse a localizaciones dentro de la secuencia de comportamiento del caso de uso. El beneficio neto es que los nombres del punto de extensión sirven como etiquetas para los estados, las declaraciones, y las regiones dentro de la secuencia de comportamiento. Esto no significa que deban estar presentes en el texto de la secuencia de comportamiento original.

Los puntos de extensión son nombres que permiten separar la localización de la opción para realizar una inserción. Una herramienta de edición puede poner un punto de extensión en un recubrimiento del texto de la secuencia de comportamiento sin modificar el texto original, o puede tener una tabla separada de los nombres de puntos de extensión distribuida en declaraciones (por número de declaración, etiquetas internas, o gráficos directos). En cualquier caso, el efecto neto corresponde a la localización de cada nombre de punto de extensión entre uno o más pasos en las secuencias de comportamiento. En el ejemplo de pseudocódigo en la Figura 13.161 un nombre de punto de extensión entre símbolos de < y > refiriéndose a una localización en la secuencia de comportamiento. Hay muchas sintaxis posibles para localizar puntos de extensión dentro de secuencias de comportamiento, dependiendo del lenguaje utilizado para especificar la secuencia. Esto es un ejemplo, pero no la única forma.



**Figura 13.161** Declaración de puntos de extensión

## punto de variación semántica

Punto de variación en la semántica de un metamodelo. Proporciona un grado de libertad deliberado para la interpretación de la semántica del metamodelo.

### Discusión

Una misma semántica de ejecución no resulta adecuada para todas las posibles aplicaciones. Los distintos lenguajes de programación y los diferentes propósitos requieren variaciones de la semántica; algunas de ellas son sutiles y otras bastas. Un punto de variación semántica es un asunto en que distintos creadores de modelos o diferentes entornos de ejecución discrepan con respecto a la semántica específica, frecuentemente con buenas razones. Al identificar y dar nombre a los puntos de variación semántica, se pueden evitar las discusiones acerca de la semántica "correcta" de un sistema.

Por ejemplo, la opción de permitir o no la clasificación múltiple o la clasificación dinámica es un punto de variación semántica. Cada opción es una variación semántica.

Entre otros ejemplos de puntos de variación semántica se incluye si una llamada puede proporcionar más de un valor y si las clases existen durante el tiempo de ejecución como objetos reales.

## realización

Relación entre una especificación y su implementación; indicación de la herencia de comportamiento sin la herencia de estructura.

*Véase también* interfaz.

### Semántica

Las especificaciones describen el comportamiento o estructura de algo sin determinar cómo se implementará el comportamiento. Una implementación proporciona los detalles relativos a la forma de implementar el comportamiento de una forma computable efectivamente. La relación entre un elemento que especifica comportamiento y otro que proporciona una implementación se denomina realización. En general, hay muchas maneras de realizar una especificación. De forma similar, un elemento puede realizar más de una especificación. La realización es, por tanto, una relación muchos-a-muchos entre elementos.

El significado de la realización es que el elemento cliente tiene que admitir todo el comportamiento del elemento proveedor pero no tiene por qué satisfacer su estructura o su implementación. Un clasificador cliente, por ejemplo, tiene que admitir las operaciones del clasificador proveedor; además tiene que admitir todas las máquinas de estados que especifiquen el comportamiento externo del proveedor. Pero todos los atributos, asociaciones, métodos o máquinas de estados del proveedor que especifiquen la implementación son irrelevantes para el cliente. Observe que el cliente no hereda realmente las operaciones del proveedor. Tiene que declararlas él mismo o heredarlas de algún antecesor para que queden cubiertas todas las operaciones del pro-

veedor. En otras palabras, el proveedor de una realización indica qué operaciones tienen que estar presentes en el cliente, pero el cliente es quien tiene la responsabilidad de aportarlas.

Ciertas clases de elementos, tal como las interfaces y los casos de uso, están destinados a especificar el comportamiento y no contienen información de implementación. Otras clases de elementos, tales como las clases, están destinadas a implementar el comportamiento. Contienen información de implementación pero también se puede utilizar de forma más abstracta como especificadores. Normalmente, la realización relaciona un elemento de especificación, tal como un caso de uso o una interfaz, con un elemento de implementación, tal como una colaboración o una clase. Es posible utilizar un elemento de implementación, tal como una clase, para la especificación. Se puede situar en el lado de especificación de una relación de realización. En este caso, sólo las partes de especificación de la clase proveedora afectarán al cliente. Entonces, hablando con más precisión, la realización es una relación entre dos elementos en los cuales las partes de especificación de comportamiento externo de uno de ellos restringen la implementación del otro. Se podría pensar que se trata de una herencia de comportamiento sin herencia de estructura o de implementación (y con la necesidad de declarar realmente las operaciones por parte del cliente).

Si el elemento de especificación es una clase abstracta sin atributos, sin asociaciones y únicamente con operaciones abstractas, entonces toda especialización de la clase abstracta realiza la clase abstracta, porque nada hay que heredar salvo la especificación.

El elemento que hace la implementación tiene que admitir todo el comportamiento incluido en el elemento que hace la especificación. Por ejemplo, una clase tiene que contener todas las operaciones de aquellas interfaces que realiza, con una semántica que sea consistente con todas las especificaciones que requieran las interfaces. La clase puede implementar operaciones adicionales y la implementación de las operaciones puede hacer cosas adicionales, siempre y cuando no se violen las especificaciones de operación de las interfaces.

## Notación

La relación de realización se muestra mediante una ruta discontinua con una punta de flecha triangular hueca adyacente al elemento que proporciona la especificación y con el origen en el elemento que proporciona la implementación (Figura 13.162).

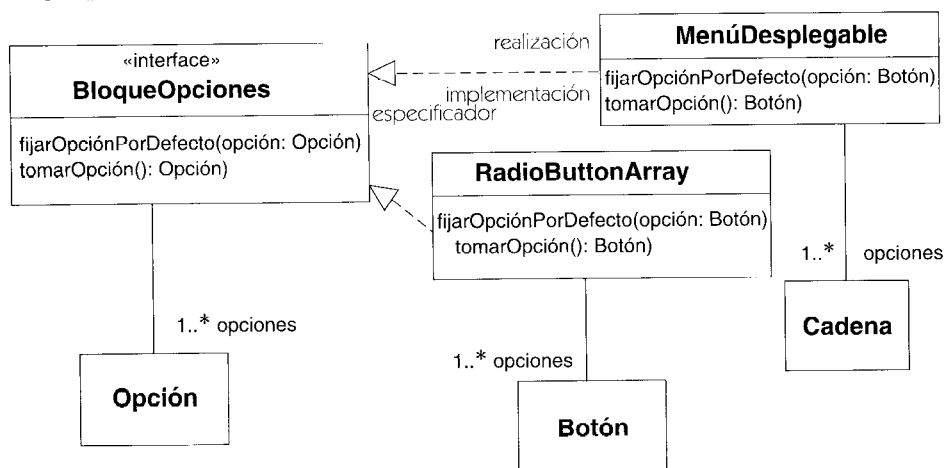


Figura 13.162 Relación de realización

## Discusión

Otro caso importante es la realización de un caso de uso mediante una colaboración (Figura 13.163). Los casos de uso especifican secuencias de funcionalidad y comportamiento visibles externamente pero no proporcionan una implementación. Una colaboración describe los objetos que implementan el comportamiento del caso de uso y la forma en que interaccionan para hacerlo. Normalmente, una colaboración implementa un caso de uso, pero una colaboración se puede implementar empleando colaboraciones subordinadas, cada una de las cuales hace una parte del trabajo. Los objetos y las clases que se utilizan para implementar una colaboración suele aparecer también en otras colaboraciones.



Figura 13.163 Realización de un caso de uso por colaboración

Toda clase de la colaboración dedica una parte de su funcionalidad al caso de uso que se está implementando. Por tanto, el caso de uso será implementado eventualmente por partes en varias clases.

## realizar

Proporcionar la implementación de un elemento de especificación.

*Véase realización.*

## recepción

La declaración de que un clasificador está preparado para reaccionar cuando reciba una señal. Es un miembro de un clasificador.

## Semántica

Una recepción es una declaración que indica que un clasificador está preparado para admitir una instancia de una señal y reaccionar frente a ella. Una recepción es algo parecido a una operación. Declara la signatura de un mensaje que admite el clasificador y especifica su significado.

## Estructura

Una recepción posee las propiedades siguientes:

|              |                                                                                                                                                                               |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| polimorfismo | Denota si la respuesta del clasificador frente a la señal es o no siempre la misma. Se codifica mediante la propiedad <b>isPolymorphic</b> , mediante los valores siguientes: |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                |                                                                                                                                                                                                                           |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>true</b>    | La respuesta es polimorfa: puede depender del estado y también puede ser anulada por un descendiente.                                                                                                                     |
| <b>false</b>   | La respuesta debe ser la misma, independientemente del estado, y no puede ser anulada por un descendiente. El efecto neto es que tiene que haber una única transición de toda la máquina de estados que maneja el evento. |
| señal          | Denota la señal a la cual responderá el clasificador.                                                                                                                                                                     |
| especificación | Una expresión que enuncia los efectos que causa la recepción de la señal.                                                                                                                                                 |

Notación

La recepción puede mostrarse en la lista de operaciones de una clase o interfaz empleando la sintaxis correspondiente a una operación con la palabra reservada «**signal**» delante del nombre de la señal.

Alternativamente, se puede situar una lista de firmas de señal en su propio compartimento; el compartimento tiene el nombre **Señales**. Se muestran las dos formas en la Figura 13.164.

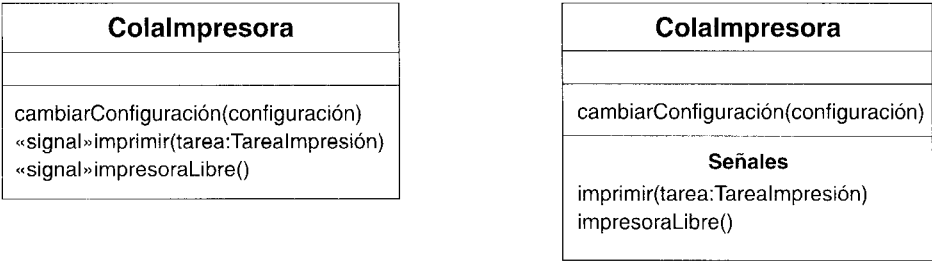


Figura 13.164 Hay dos maneras de indicar la recepción de una señal

receptor

Dícese de aquel objeto que maneja una instancia de mensaje que se le pasa desde un objeto emisor.

recibir

Manejar una instancia de mensaje que se nos pasa procedente de un objeto emisor.

Véase emisor, receptor.

referencia

Alude a una forma de denotar un elemento del modelo; suele llamarse puntero.



## Semántica

Los elementos del modelo están conectados mediante dos metarrelaciones: propiedad y referencia. La propiedad es la relación que existe entre un elemento y sus partes constitutivas, las partes que están definidas en su interior y que él posee. La relación de propiedad forma un árbol estricto. Los elementos contenidos están subordinados al elemento contenedor. La propiedad, el control de configuración y el almacenamiento de modelos están basados en la jerarquía de contención.

La referencia es una relación entre elementos de un mismo nivel de detalle o bien entre diferentes contenedores. Por ejemplo, la referencia es la relación que existe entre una asociación y las clases participantes en ella, entre un atributo y la clase o tipo de dato que es una propiedad de tipo, entre una plantilla ligada y los valores de sus argumentos. Para que sea posible una referencia, el elemento del que se haga la referencia tiene que ser visible para el elemento que la hace. En general, esto significa que el paquete que contiene el origen de la referencia debe poder ver el paquete que contiene el destino de la referencia. Esto requiere una relación adecuada de acceso o de importación entre los paquetes. También requiere que el elemento al que se haga referencia tenga una configuración de visibilidad que le permita ser visto fuera de su paquete, a no ser que la referencia tenga lugar en el mismo paquete.

Obsérvese que la referencia es una relación interna de metamodelo, no una relación visible por el usuario; se emplea para construir las demás relaciones.

## refinamiento

Denota una relación que representa una especificación más completa de algo que ya se ha especificado con un cierto nivel de detalle o en un nivel semántico diferente.

*Véase también* abstracción.

## Semántica

El refinamiento es una conexión histórica o computable entre dos elementos que poseen una correspondencia (no necesariamente completa) entre sí. Con frecuencia, los dos elementos se encuentran en modelos diferentes. Por ejemplo, una clase de diseño puede ser un refinamiento de una clase de análisis; tiene los mismos atributos lógicos pero sus clases pueden provenir de una biblioteca de clases específica. Sin embargo, un elemento puede refinar a otro en un mismo modelo. Por ejemplo, una versión optimizada de una clase es un refinamiento de la versión sencilla pero ineficiente de esa misma clase. La relación de refinamiento puede contener una descripción de la correspondencia, que se puede escribir en un lenguaje formal (tal como OCL o algún lenguaje lógico o de programación). También puede ser un texto informal (lo cual, evidentemente, impide toda computación automática pero puede ser útil en las fases iniciales del desarrollo). Se puede utilizar el refinamiento para modelar la elaboración paso a paso del desarrollo, optimización, transición y el marco de trabajo.

## Estructura

El refinamiento es una clase de dependencia. Relaciona a un cliente (el elemento que está más desarrollado) con un proveedor (el elemento que es la base del refinamiento).

## Notación

Los refinamientos se indican mediante una flecha de dependencia (una flecha discontinua con la punta en el elemento más general y el origen en el elemento más específico) que posee la palabra reservada «**refine**». La correspondencia puede estar asociada a la ruta de dependencia mediante una línea discontinua conectada a una nota. Se han propuesto diferentes tipos de refinamientos; es posible indicarlos mediante más estereotipos. En muchos casos, el refinamiento conecta elementos de diferentes modelos y, por tanto, no será visible gráficamente. Con cierta frecuencia, sólo está implícito.

## Ejemplo

La optimización es una clase típica de refinamiento. La Figura 13.165 muestra un tablero de ajedrez que posee una sencilla representación en el modelo de análisis, pero otra representación mucho más elaborada y oscura en el modelo de diseño. La clase de diseño no es una especialización de la clase de análisis, porque tiene una forma completamente diferente. La clase del modelo de análisis y la del modelo de diseño tienen el mismo nombre, porque representan un mismo concepto en dos niveles semánticos diferentes.

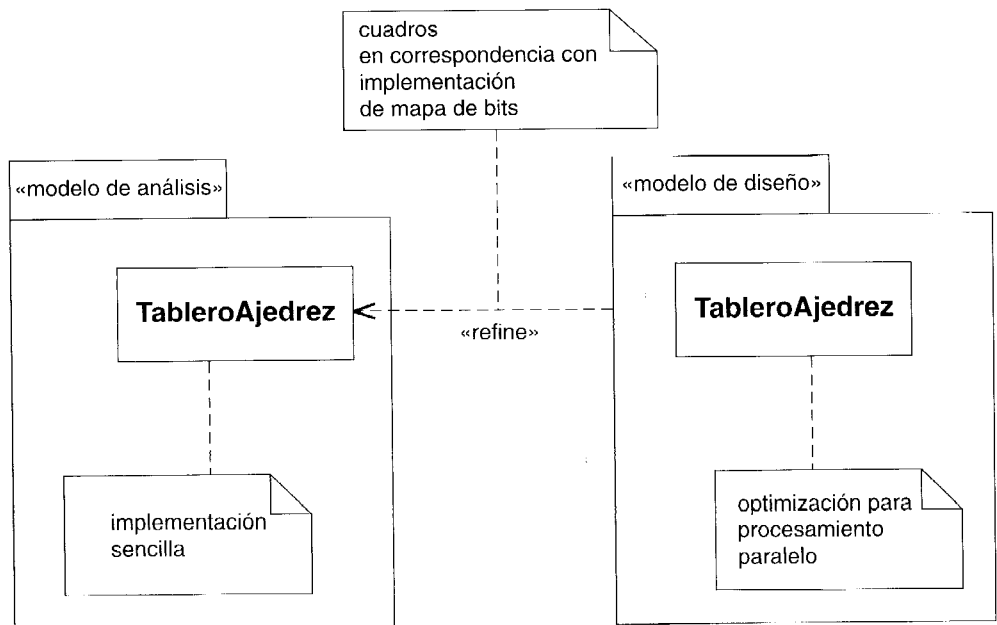


Figura 13.165 Refinamiento